TEMCO LEARN

Semantic tagging

1.0 · 14 pages · built Nov 13, 2026

1. Semantic tagging
2. Semantic tagging on Temco hardware
3. Haystack v2 — Current Implementation
4. T3000 Haystack & Brick Auto-Tagging + MCP — Design Document
5. T3000 MCP Server — Design
6. Haystack v1 — Legacy Single-Table Design
7. mcp-claude-desktop
8. mcp-vscode-copilot
9. MCP API Examples
10. AI Integration — Local LLM Server
11. AI Integration — Cloud LLM APIs
12. AI Chat Page — Integration Design
13. FDD — Native Fault Detection & Diagnostics for T3000
14. The OpenBIM and Haystack plan

Semantic tagging

Haystack tags and the OpenBIM plan: how a point says what it measures, how the tags get filled in, and how an assistant reads them over MCP.

Contents

- Semantic tagging on Temco hardware
- Haystack v2 — Current Implementation
- T3000 Haystack & Brick Auto-Tagging + MCP — Design Document
- T3000 MCP Server — Design
- Haystack v1 — Legacy Single-Table Design
- mcp-claude-desktop
- mcp-vscode-copilot
- MCP API Examples
- AI Integration — Local LLM Server
- AI Integration — Cloud LLM APIs
- AI Chat Page — Integration Design
- FDD — Native Fault Detection & Diagnostics for T3000
- The OpenBIM and Haystack plan

Semantic tagging on Temco hardware

T3000 does not have to learn a second name for a point. A point keeps the name it was given, and carries tags beside it: what it measures, what it belongs to, and which equipment it is part of. Those tags are what makes a flat point list readable by a machine — by our own assistant, by Claude or Copilot through MCP, and by a fault-detection rule that has to know which sensor it is looking at.

This is a subject that crosses products: the tags are stored on the controller, served by the web front end, and consumed by anything that speaks MCP.

The order to read it in

1. **How tagging works now** — the current implementation, and the shape of the data.
2. **Auto-tagging** — the rules that fill tags in from a point's own name.
3. **The MCP server** — how an assistant asks for the semantic model and gets it back.
4. **The legacy design** — the single-table version, kept for the controllers still on it.
5. **The assistant and the rules** — the local model, its API, the chat screen, and the FDD engine that reads the same tags.

The plan

The last chapter is the strategic plan this work is being built against: OpenBIM (IFC through Bonsai and thatOpen) together with Haystack, across the C++ front end, the web front end and the firmware.

Haystack v2 — Current Implementation

1. Database Schema

1.1 HAYSTACK_TAGS — Tag Dictionary

```sql
-- Migration: m20260607_add_haystack_v2.rs
CREATE TABLE IF NOT EXISTS HAYSTACK_TAGS (
    tag_name   TEXT PRIMARY KEY,
    doc        TEXT,
    category   TEXT NOT NULL DEFAULT 'custom',  -- 'haystack' | 'custom'
    deprecated INTEGER NOT NULL DEFAULT 0,
    source     TEXT DEFAULT 'user'              -- 'user' | 'v4'
);
```

Populated by:

- `POST /api/haystack/sync` — fetches standard tags from `https://project-haystack.org/download/defs.json`, parses `rows[].def.val` → `tag_name`, `rows[].doc` → `doc`, `rows[].is` → parent relations
- Frontend Standard Tags page "Sync" button calls this endpoint
- Custom tags created via `POST /api/haystack/tags` or the Custom Tags page

1.2 HAYSTACK_TAG_RELATIONS — Tag Inheritance

```sql
CREATE TABLE IF NOT EXISTS HAYSTACK_TAG_RELATIONS (
    tag_name   TEXT NOT NULL,
    parent_tag TEXT NOT NULL,
    PRIMARY KEY (tag_name, parent_tag)
);
```

Populated during sync. Example: `temp` → parent `point`, `outsideAirTemp` → parent `air`, `temp`, `point` (multi-parent).

1.3 HAYSTACK_POINT_TAGS — Point ↔ Tag Mapping

```SQL
CREATE TABLE IF NOT EXISTS HAYSTACK_POINT_TAGS (
    serial_number INTEGER NOT NULL,
    point_type    TEXT NOT NULL,    -- 'INPUT' | 'OUTPUT' | 'VARIABLE'
    point_index   TEXT NOT NULL,    -- 1-based index as string
    point_id      TEXT NOT NULL,    -- 'dev237219.in5'
    tag_name      TEXT NOT NULL,
    PRIMARY KEY (serial_number, point_type, point_index, tag_name)
);

CREATE INDEX IF NOT EXISTS idx_hpt_serial ON HAYSTACK_POINT_TAGS (serial_number);
CREATE INDEX IF NOT EXISTS idx_hpt_tag ON HAYSTACK_POINT_TAGS (tag_name);
```

One row per tag per point. Multiple tags = multiple rows. Primary key prevents duplicates.

---

2. Auto-Tagging Engine

File: `api/src/t3_device/haystack_tags_service.rs` **Function:** `auto_tag_point()`

2.1 Triggers

Trigger	When
FFI sync (`t3_ffi_sync_service.rs`)	Every INPUTS/OUTPUTS/VARIABLES row inserted or updated
`POST /api/haystack/rebuild`	Manual trigger — clears all tags for given serials, re-tags all points

2.2 Logic

TEXT

```
auto_tag_point(db, point_table, serial_number, point_index, label, digital_analog, units
  │
  ├─ 1. Normalize point_table → point_type (INPUTS→INPUT, OUTPUTS→OUTPUT, VARIABLES→VARI
  ├─ 2. Build point_id: "dev{serial}.{in|out|var}{index}"
  ├─ 3. Check if point already has tags → skip (don't override manual assignments)
  ├─ 4. Base tag: "point" (all points)
  ├─ 5. Type-based tags:
  │     INPUT     → add "sensor"
  │     OUTPUT    → digital_analog==1 → add "cmd"
  │               → digital_analog!=1 → add "actuator"
  ├─ 6. Units-based tags (lowercase substring match):
  │     deg.c / celsius       → "temp", "degC"
  │     deg.f / fahrenheit    → "temp", "degF"
  │     %rh / humidity        → "humidity"
  │     ppm                   → "co2"
  │     pa / pascal           → "pressure"
  │     cfm                   → "air", "flow"
  │     kw                    → "power"
  │     volt                  → "voltage"
  │     amp / ma              → "current"
  │     % (standalone)        → "percent"
  │     fpm                   → "air", "velocity"
  └─ 7. Label-based tags (lowercase substring match):
        temp                   → "temp" (if not already added)
        setpoint / "sp "       → "setpoint"
        alarm / fault          → "alarm"
        status / "run "        → "status", "run"
        enable                 → "enable"
        damper                 → "damper"
        valve                  → "valve"
        fan                    → "fan"
        pump                   → "pump"
        supply / discharge     → "discharge"
        return / exhaust       → "return"
        outside / oat          → "outside"
```

2.3 Rebuild

Function: `rebuild_tags_for_serials(db, serial_numbers)`

1. Delete all rows from `HAYSTACK_POINT_TAGS` for given serials
2. Read all INPUTS, OUTPUTS, VARIABLES for those serials
3. Call `auto_tag_point()` on every row
4. Return count of points tagged

2.4 Search

Function: `search_points(db, SearchPointsRequest)`

Filters: `device_serials`, `point_types`, `tag_filter`, `label_filter`, `units_filter`. Returns matching `PointTagEntry` rows.

---

3. REST API Routes

File: `api/src/t3_device/haystack_tags_routes.rs` **Registered in:** `api/src/server.rs` via `create_haystack_tags_routes()`

Route	Method	Handler	Description
`/api/haystack/tags`	GET	`list_tags`	List all tags. Query: `?filter=haystack|custom`
`/api/haystack/tags`	POST	`create_tag`	Create custom tag. Body: `{tagName, doc?}`
`/api/haystack/tags/:name`	PUT	`update_tag`	Update doc or deprecated flag. Body: `{doc?, deprecated?}`
`/api/haystack/tags/:name`	DELETE	`delete_tag_handler`	Delete tag (custom only, must be unused)
`/api/haystack/tag-tree`	GET	`get_tag_tree`	Tag hierarchy. Returns `TagTreeNode[]` with children
`/api/haystack/point-tags/read`	POST	`read_point_tags`	Body: `{serialNumbers, pointType?}`
`/api/haystack/point-tags/write`	POST	`write_point_tags`	Body: `BatchPointTagUpdate[]` with `serial_number, point_type, point_index, point_id, add_tags?, remove_tags?, set_tags?`
`/api/haystack/replace-tag`	POST	`replace_tag`	Body: `{oldTag, newTag}`. Updates all point references
`/api/haystack/rebuild`	POST	`rebuild_tags`	Body: `{serialNumbers}`. Re-runs auto-tagging
`/api/haystack/sync`	POST	`sync_official_tags`	Fetches `project-haystack.org/defs.json`, reseeds standard tags

All routes use local SQLite (`local_config_conn`) — never the center DB.

4. Rust Code Structure

```
api/
  ├── migration/src/
  │   ├── m20260607_add_haystack_v2.rs       ← Creates 3 tables + indexes
  │   └── m20260607_remove_haystack_v1.rs    ← DROP TABLE HAYSTACK_ENTITY
  │
  └── src/
      ├── server.rs                           ← Registers haystack routes (line 217-218)
      ├── entity/t3_device/
      │   └── haystack_entity.rs              ← SeaORM entity (deprecated — v1 table dro
      └── t3_device/
          ├── mod.rs                          ← pub mod declarations
          ├── haystack_tags_service.rs         ← Service: CRUD, tree, auto-tag, rebuild,
          ├── haystack_tags_routes.rs          ← Axum route handlers (10 endpoints)
          └── t3_ffi_sync_service.rs          ← Calls auto_tag_point() during FFI sync (
```

FFI integration (`t3_ffi_sync_service.rs`): After each INPUTS/OUTPUTS/VARIABLES row is inserted or updated, calls `auto_tag_point()` with the point's label, digital_analog, and derived units.

---

5. Frontend

5.1 Store

File: `src/t3-react/features/haystack/store/haystackStore.ts` **Library:** Zustand

State: `tags`, `tagTree`, `pointTags`, `isLoading`, `error`, `selectedTag` Actions: `fetchTags`, `fetchTagTree`, `fetchPointTags`, `createTag`, `updateTag`, `deleteTag`, `replaceTag`, `batchUpdatePointTags`, `setSelectedTag`

5.2 Pages

URL	Component	File
`/t3000/haystack-tags`	`HaystackTagsPage`	`features/haystack/pages/HaystackTagsPage.tsx`
`/t3000/custom-tags`	`CustomTagsPage`	`features/haystack/pages/CustomTagsPage.tsx`

HaystackTagsPage: Standard tags (`category=haystack`). Interactive tree (expand/collapse/filter). "Sync" button → `POST /api/haystack/sync`.

CustomTagsPage: Create/edit/delete custom tags. Inline editing. Delete confirmation. Only custom tags can be deleted (standard tags protected server-side).

5.3 Tags Column (Input/Output/Variable Pages)

Components:

- `TagsColumnCell.tsx` — Renders tags inline in the data grid
- Tag Drawer — Opens on click, allows adding/removing tags per point

Data flow:

1. Page loads → `fetchTagsForDevice(serialNumber)` → `POST /api/haystack/point-tags/read`
2. Tags column renders chip-style inline tags (max 2 visible, `+N` overflow)
3. Click tag cell → drawer opens with full tag list + add/remove controls
4. Save → `POST /api/haystack/point-tags/write`

5.4 Menu & Routes

Menu config: `src/t3-react/config/menuConfig.ts`

- "Haystack Tags" → `#/t3000/haystack-tags` (Alt+Y)
- "Haystack Custom Tags" → `#/t3000/custom-tags`

Routes: `src/t3-react/app/router/routes.ts`

- `/t3000/haystack-tags` → `HaystackTagsPage`
- `custom-tags` nested route → `CustomTagsPage` (in `App.tsx`)

PageHeader: `src/t3-react/layout/PageHeader.tsx`

- `/t3000/haystack-tags` → breadcrumb: "Haystack"
- `/t3000/custom-tags` → breadcrumb: "Custom Tags" under "Haystack"

---

6. API Request/Response Examples

List Tags

```TEXT
GET /api/haystack/tags?filter=haystack
→ { "tags": [{ tag_name, doc, category, deprecated, source, usage_count, parents }], "to
```

Read Point Tags

```
TEXT

POST /api/haystack/point-tags/read  { "serialNumbers": "237219" }
→ { "entries": [{ serial_number, point_type, point_index, point_id, tag_name }], "total"
```

Write Point Tags

```
TEXT

POST /api/haystack/point-tags/write  [{ serialNumber, pointType, pointIndex, pointId, se
→ { "message": "Point tags updated", "count": 1 }
```

Sync

```
TEXT

POST /api/haystack/sync
→ { "success": true, "message": "Standard tags synced...", "count": 243 }
```

Rebuild

```
TEXT

POST /api/haystack/rebuild  { "serialNumbers": [237219] }
→ { "success": true, "message": "Haystack tags rebuilt", "updated": 1, "pointsTagged": 1
```

---

7. Related

Document	Description
v1-haystack-legacy-single-table.md	Deprecated v1 design
v3-haystack-auto-tagging-mcp.md	Planned v3: rules engine + MCP server

T3000 Haystack & Brick Auto-Tagging + MCP — Design Document

1. Overview

1.1 Why

T3000 devices expose raw BACnet points: `Device 1001, AI-5, name="OAT", value=72.3`. Humans and LLMs cannot understand what this point represents without additional context. Vendor platforms (JCI OpenBlue, Honeywell Forge, Siemens Building X) each provide their own proprietary semantic layers, locked inside their ecosystems. Independent MSIs running mixed-vendor portfolios have no standard way to make their building data AI-readable.

1.2 Purpose

Add a semantic tagging layer on top of T3000's existing point data that:

- Automatically assigns Haystack tags (`[outside, air, temp]`) and Brick classes (`Outside_Air_Temperature_Sensor`) to every device point
- Exposes tagged data to LLM agents via MCP, enabling natural language queries like "Why is AHU-3 not cooling?"
- Keeps all logic local (no cloud dependency, no credentials egress)

1.3 Concepts Used

Concept	Source	Role
Haystack	Project Haystack (2011)	Flat tag vocabulary — `point` , `sensor` , `outside` , `air` , `temp`
Brick	Brick Schema (2016, UC Berkeley)	Formal class hierarchy — `Outside_Air_Temperature_Sensor isA Sensor isA Point`
YAML Rules	brick-bacnet-mcp (Yves Habchy, 2026)	Regex patterns mapping point names → tags + classes
MCP	Model Context Protocol (Anthropic, 2024)	JSON-RPC/stdio protocol for LLM agent tool access

1.4 How It Integrates With T3000

T3000 v2 already has Haystack tagging (manual assignment, basic auto-tagging). V3 adds:

TEXT

```
v2 (current)                          v3
──────────                            ──
Hardcoded auto_tag_point()      →     Rules table + regex engine
Haystack tags only              →     + Brick class per point
POST /api/haystack/rebuild      →     /api/haystack/auto-tagging/run
No MCP                          →     MCP server (7 tools over stdio)
No rules management UI          →     Auto-Tagging & MCP page (3 tabs)
```

1.5 End-to-End Flow

TEXT

```
1. DEVICE SYNC
   C++ FFI → INPUTS/OUTPUTS/VARIABLES tables updated

2. AUTO-TAGGING (three-step engine)
   Step 1 — Range rules: lookup (point_type, digital_analog, Range_Field) → baseline tag
   Step 2 — Haystack regex: eval_rules(label, units) → extra tags (INSERT OR IGNORE)
   Step 3 — Brick regex: eval_rules(label, units) → brick_class (overrides range)
   → Write tags to HAYSTACK_POINT_TAGS (auto_assigned=1)
   → Write brick_class to HAYSTACK_POINT_BRICK_CLASS

3. WEB UI
   Auto-Tagging & MCP page → manage rules, run tagging, view MCP status
   Inputs/Outputs/Variables pages → Tags column shows tags + Brick badge

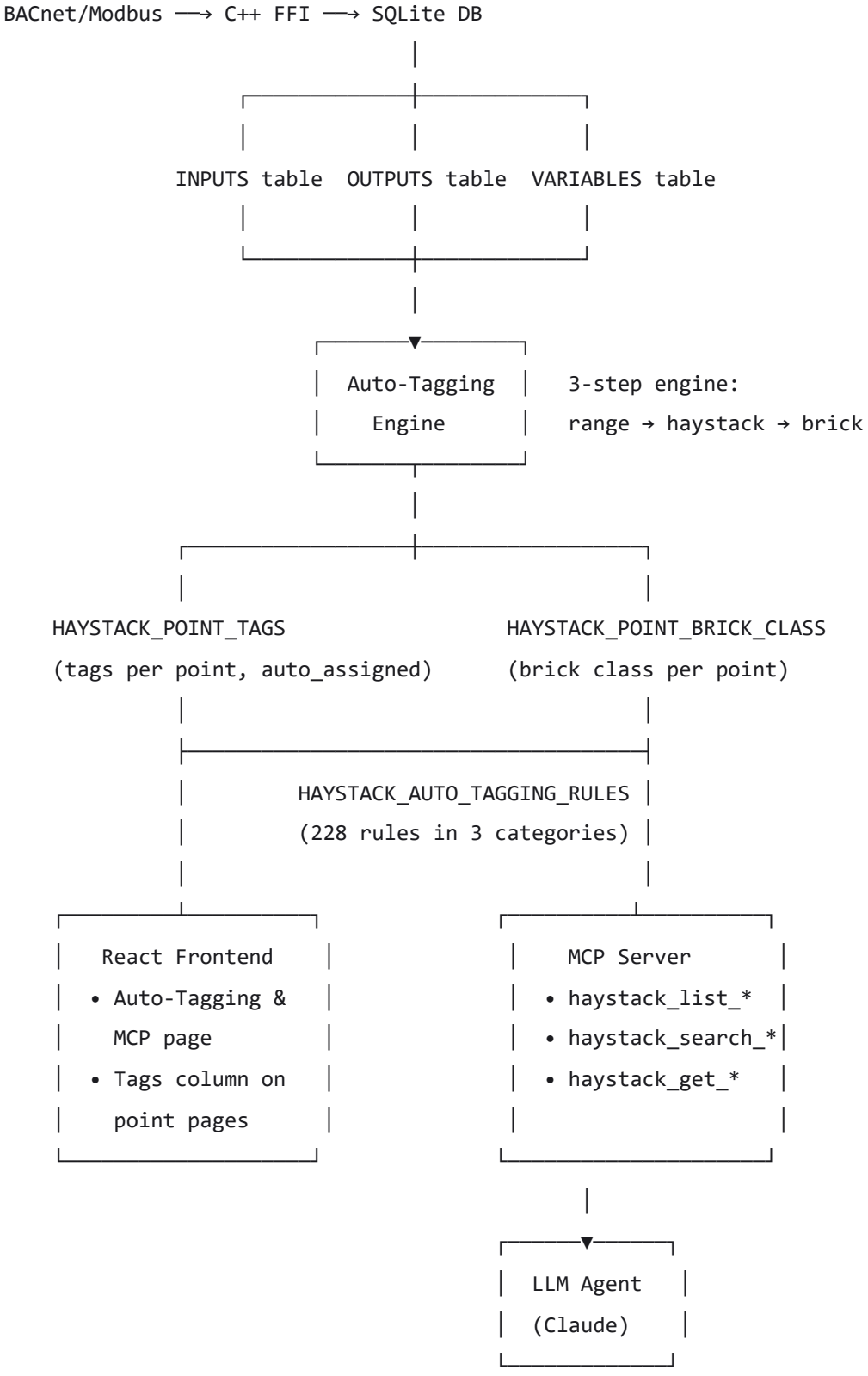
4. MCP SERVER
   Tools query HAYSTACK_POINT_TAGS + HAYSTACK_POINT_BRICK_CLASS + point tables
   Returns semantic-tagged data via JSON-RPC/stdio

5. LLM AGENT
   User: "What's the outside air temperature?"
   Agent → get_tagged_topology(filter_brick="Outside_Air_Temperature_Sensor")
        → Returns: 72.3°F
```

---

2. Architecture Diagram

TEXT



3. Database Schema

3.1 `HAYSTACK_POINT_TAGS` — tag assignments (1 row per tag per point)

```sql
CREATE TABLE HAYSTACK_POINT_TAGS (
    serial_number  INTEGER NOT NULL,
    point_type     TEXT NOT NULL,
    point_index    TEXT NOT NULL,
    point_id       TEXT NOT NULL,
    tag_name       TEXT NOT NULL,
    auto_assigned  INTEGER NOT NULL DEFAULT 0,  -- 1=auto-tagged, 0=manual
    PRIMARY KEY (serial_number, point_type, point_index, tag_name)
);
```

- `auto_assigned` added by m20260716; default 0 preserves existing manual tags
- Auto-tagging INSERTs with `auto_assigned=1`
- Reset deletes `WHERE auto_assigned=1` — manual tags survive
- Old `brick_class` column dropped in m20260716

3.2 `HAYSTACK_POINT_BRICK_CLASS` — brick classification (1 row per point)

```sql
CREATE TABLE HAYSTACK_POINT_BRICK_CLASS (
    serial_number INTEGER NOT NULL,
    point_type    TEXT NOT NULL,
    point_index   INTEGER NOT NULL,
    brick_class   TEXT NOT NULL,
    PRIMARY KEY (serial_number, point_type, point_index)
);
```

- Separate from `HAYSTACK_POINT_TAGS` — brick_class is point-level, not tag-level
- Replaces old `__brick_class__` marker row pattern from v3.0
- Preview joins with `LEFT JOIN HAYSTACK_POINT_BRICK_CLASS`

3.3 `HAYSTACK_AUTO_TAGGING_RULES` — rule definitions

Three categories share one table (haystack, brick, range):

SQL

```sql
CREATE TABLE HAYSTACK_AUTO_TAGGING_RULES (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    rule_name       TEXT NOT NULL UNIQUE,
    category        TEXT NOT NULL CHECK(category IN ('haystack','brick','range')),

    -- Regex rules (haystack / brick)
    pattern         TEXT,           -- regex (NULL for range rules)
    units           TEXT,           -- units filter or display unit
    object_types    TEXT,           -- object type filter or range category

    -- Range rules (range)
    point_type      TEXT,           -- 'INPUT'|'OUTPUT'|'VARIABLE' (NULL for regex)
    digital_analog INTEGER,         -- 0=digital, 1=analog (NULL for regex)
    range_value     INTEGER,        -- matches Range_Field (NULL for regex)

    -- Output (all categories)
    haystack_tags TEXT,             -- comma-separated
    brick_class    TEXT,            -- Brick class name
    haystack_kind TEXT,             -- Haystack kind tag
    haystack_unit TEXT,             -- Normalized Haystack unit

    -- Control
    enabled        INTEGER NOT NULL DEFAULT 1,
    priority       INTEGER NOT NULL DEFAULT 0,
    created_at     TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at     TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Category usage matrix:

Column	haystack	brick	range
`pattern`	regex	regex	NULL
`units`	optional filter	optional filter	display unit (e.g. "Amps")
`object_types`	optional filter	optional filter	range category (e.g. "Temperature")
`point_type`	NULL	NULL	INPUT/OUTPUT/VARIABLE
`digital_analog`	NULL	NULL	0 or 1
`range_value`	NULL	NULL	0–104
`haystack_tags`	comma-separated	NULL	comma-separated
`brick_class`	NULL	class name	class name

3.4 Seed Data

Category	Count	Source
`haystack` (regex)	31	brick-bacnet-mcp YAML
`brick` (regex)	37	brick-bacnet-mcp YAML
`range` (metadata)	~160	Frontend rangeData.ts
Total	**~228**	

Rule examples:

TEXT

```
rule_name:    brick:oat          category: brick
pattern:      (?i)(oat\|outside[_ ]?air[_ ]?temp)
brick_class:  Outside_Air_Temperature_Sensor

rule_name:    hs:oat             category: haystack
pattern:      (?i)(oat\|outside[_ ]?air[_ ]?temp)
haystack_tags: point,sensor,outside,air,temp

rule_name:    range:in-ana-3     category: range
point_type:   INPUT              digital_analog: 1    range_value: 3
haystack_tags: point,sensor,air,temp
brick_class:  Temperature_Sensor
units:        Deg.C
```

---

4. Rule Engine

File: `api/src/haystack/auto_tagging_service.rs`

4.1 Evaluation — Three-Step Flow

Auto-tagging runs in three sequential steps per point. Range rules apply first (metadata-based), then regex rules refine.

TEXT

```
For each point (row from INPUTS/OUTPUTS/VARIABLES):
  Read: SerialNumber, point_index, Full_Label, Label, Units,
        Digital_Analog, Range_Field

  display_label = Full_Label || Label || ""

  ┌── STEP 1: Range Rules ──────────────────────────┐
  │  SELECT * FROM HAYSTACK_AUTO_TAGGING_RULES      │
  │  WHERE category='range' AND enabled=1           │
  │    AND point_type=? AND digital_analog=?        │
  │    AND range_value=?                            │
  │                                                 │
  │  → INSERT tags into HAYSTACK_POINT_TAGS         │
  │    (auto_assigned=1, INSERT OR IGNORE on PK)    │
  │  → INSERT OR REPLACE brick_class into           │
  │    HAYSTACK_POINT_BRICK_CLASS                   │
  └─────────────────────────────────────────────────┘

  ┌── STEP 2: Haystack Regex Rules ─────────────────┐
  │  eval_rules(display_label, units, None,         │
  │             haystack_rules)                     │
  │  → INSERT OR IGNORE extra tags (auto_assigned=1)│
  │  → IGNORE prevents overwriting Step 1 tags      │
  └─────────────────────────────────────────────────┘

  ┌── STEP 3: Brick Regex Rules ────────────────────┐
  │  eval_rules(display_label, units, None,         │
  │             brick_rules)                        │
  │  → INSERT OR REPLACE brick_class                │
  │  → REPLACE = regex overrides range-based class  │
  └─────────────────────────────────────────────────┘

  tagged++ if any step produced output
```

4.2 `eval_rules()` — Regex Matching

```rust
fn eval_rules(label, units, object_type, rules) -> Option<&CompiledRule> {
    for rule in rules {  // priority-ordered, first match wins
        if !rule.regex.is_match(label)      { continue; }
        if rule.units_filter is set
           && point.units not in filter     { continue; }
        if rule.object_types_filter is set
           && point.type not in filter      { continue; }
        return Some(rule);
    }
    None
}
```

4.3 Triggers

Trigger	Code Path
Manual "Run Auto-Tag"	`POST /api/haystack/auto-tagging/run` → `run_auto_tagging()`
"Preview" button	`POST /api/haystack/auto-tagging/preview` → `preview_auto_tagging()`
"Reset Tags" button	`POST /api/haystack/auto-tagging/reset` → `reset_auto_tags()`

5. API Routes

5.1 Existing (Unchanged from v2)

All v2 routes remain under `/api/haystack/` — tags CRUD, point-tags read/write, tag-tree, replace-tag, sync. See v2-haystack-current-implementation.md §3.

5.2 Auto-Tagging Routes

File: `api/src/haystack/auto_tagging_routes.rs`

Route	Method	Input	Output
`/api/haystack/auto-tagging/rules`	GET	Query: `?category=haystack|brick|range`	`{ rules: TaggingRule[], total: N }`
`/api/haystack/auto-tagging/rules`	POST	`{ rule_name, category, pattern?, units?, object_types?, haystack_tags?, brick_class?, haystack_kind?, haystack_unit?, point_type?, digital_analog?, range_value?, priority? }`	`{ id, message }`
`/api/haystack/auto-tagging/rules/:id`	PUT	Partial rule fields	`{ message }`
`/api/haystack/auto-tagging/rules/:id`	DELETE	—	`{ message }`
`/api/haystack/auto-tagging/rules/:id/toggle`	POST	—	`{ enabled, message }`
`/api/haystack/auto-tagging/run`	POST	`{ serialNumbers: [i32] }`	`{ success, message, tagged: N, matches: TagMatch[] }`
`/api/haystack/auto-tagging/preview`	POST	`{ serialNumbers: [i32] }`	`{ matches: TagMatch[], total: N }`
`/api/haystack/auto-tagging/reset`	POST	`{ serialNumbers: [i32] }`	`{ success, message }`

6. MCP Server

File: `api/src/haystack/mcp.rs`

6.1 Transport

- Protocol: JSON-RPC 2.0 over stdio (MCP specification)
- Startup: Embedded in Rust binary, launched with `--mcp` flag
- Auto-discovery: Registered as `t3000` MCP server

6.2 Tools

Tool	Description
`t3000_haystack_list_tags`	List all available Haystack tags
`t3000_haystack_get_point_tags`	Get tags for a specific point
`t3000_haystack_search_points`	Search points by tag or brick class filter
`t3000_haystack_auto_tag`	Trigger auto-tagging on selected devices
`t3000_haystack_preview_tags`	Preview auto-tagging results without applying
`t3000_haystack_list_rules`	List all auto-tagging rules
`t3000_haystack_get_brick_class`	Get brick class for points (queries `HAYSTACK_POINT_BRICK_CLASS`)

6.3 Tool Output Format

JSON

```
// list_objects response
{
  "device": { "serial_number": 237219, "name": "T3-XX-ESP111" },
  "points": [
    {
      "point_id": "dev237219.in5",
      "point_type": "INPUT",
      "point_index": "5",
      "label": "OAT Sensor",
      "value": 72.3,
      "units": "degF",
      "haystack_tags": ["point", "sensor", "outside", "air", "temp"],
      "brick_class": "Outside_Air_Temperature_Sensor",
      "haystack_kind": "Number",
      "haystack_unit": "°F"
    }
  ]
}
```

6.4 Agent Configuration

JSON

```json
// Claude Desktop: claude_desktop_config.json
{
  "mcpServers": {
    "t3000": {
      "command": "t3000-webview",
      "args": ["--mcp"]
    }
  }
}
```

---

7. Frontend

7.1 New Page — Auto-Tagging & MCP

Property	Value
URL	`/t3000/auto-tagging-mcp`
Component	`src/t3-react/features/haystack/pages/AutoTaggingMcpPage.tsx`
Store	Extends `haystackStore.ts` with `autoTaggingRules`, `mcpStatus` state
Menu	Haystack → Auto-Tagging & MCP

Tabs:

Tab	Content
Rules	CRUD table for all rules (haystack, brick, range), filter by category
Run Auto-Tag	Device selection, preview, run, and reset auto-tagging
MCP Server	Tool reference, connection info for LLM agents

Persistent status bar across all tabs:

TEXT

```
Auto-Tagging [ON ●——]  Device: T3-XX-ESP111  Tagged: 156/192 (81%)
[Run Auto-Tagging]  [View Coverage Report]
```

7.2 Existing Pages — Additions

Page	Change	Component
Inputs page	Tags column: add `brick_class` badge	`TagsColumnCell.tsx`
Outputs page	Same	Same
Variables page	Same	Same
Tags Drawer	Add Brick class dropdown	Tag drawer component

7.3 Menu & Routes

Menu config: `src/t3-react/config/menuConfig.ts`

```text
Haystack ▼
  ├── Standard Tags       (/t3000/haystack-tags)
  ├── Custom Tags         (/t3000/custom-tags)
  └── Auto-Tagging & MCP  (/t3000/auto-tagging-mcp)
```

Routes: `src/t3-react/app/router/routes.ts`

```text
/t3000/auto-tagging-mcp  →  AutoTaggingMcpPage
```

---

8. Rust Code Structure

TEXT

```
api/src/haystack/
  ├── mod.rs
  ├── tags_service.rs
  ├── tags_routes.rs
  ├── auto_tagging_service.rs       ← rule engine + rules CRUD + eval_rules
  ├── auto_tagging_routes.rs        ← run/preview/reset + rules CRUD endpoints
  └── mcp.rs                        ← MCP server (7 tools, JSON-RPC/stdio)

api/migration/src/
  ├── m20260715_add_auto_tagging_rules.rs
  ├── m20260716_add_point_brick_class_table.rs
  └── m20260717_add_range_rules.rs  (to be created)

src/t3-react/features/haystack/pages/
  └── AutoTaggingMcpPage.tsx        ← Rules / Run Auto-Tag / MCP Server tabs
```

---

9. Migrations

m20260715 — Original rules + brick_class column

- Creates `HAYSTACK_AUTO_TAGGING_RULES` with 68 seed rules (37 brick + 31 haystack)
- Adds `brick_class` column to `HAYSTACK_POINT_TAGS`

m20260716 — Point brick class table + auto_assigned

1. Creates `HAYSTACK_POINT_BRICK_CLASS` table
2. Migrates old `__brick_class__` marker rows → new table, then deletes them
3. Adds `auto_assigned` column to `HAYSTACK_POINT_TAGS` (DEFAULT 0)
4. Drops old `brick_class` column from `HAYSTACK_POINT_TAGS`

m20260717 — Range rules

1. Adds `point_type`, `digital_analog`, `range_value` columns to rules table
2. Drops/recreates CHECK constraint to include `'range'` category
3. Seeds ~160 range rules from frontend `rangeData.ts`

---

10. Related Documents

Document	Link
Haystack v2 Implementation Guide	v2-haystack-current-implementation.md
Haystack v1 (deprecated)	v1-haystack-legacy-single-table.md
Brick Schema	https://brickschema.org/
brick-bacnet-mcp (upstream)	https://github.com/Yveshby27/brick-bacnet-mcp

T3000 MCP Server — Design

> **Current state**: 50+ tools across 16 categories (v5 expanded from v4). This doc describes the v4 baseline (44 tools, 11 categories).

---

1. Scope

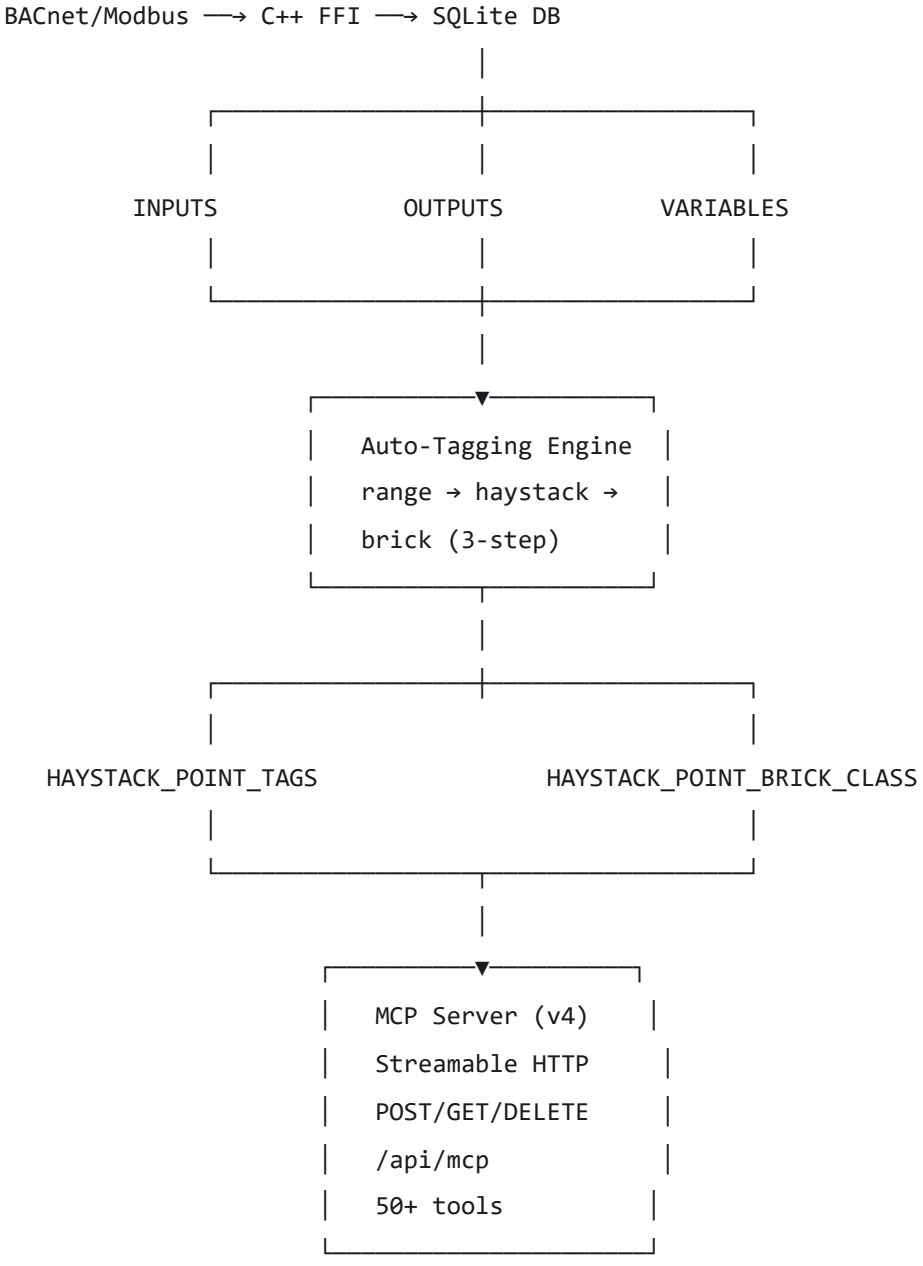
v4 extends the MCP server from 7 Haystack-only tools to 44 tools across 11 categories (since expanded to 50+ across 16 categories in v5). One file changes: `api/src/haystack/mcp.rs`. New handlers call existing public service functions — no other files touched.

	v3	v4
Tools	7	44
Categories	1 (haystack)	11
File changes	—	`mcp.rs` only

---

2. Architecture

TEXT



3. Tool Manifest

3.1 Haystack Tagging — 7 tools (existing, unchanged)

#	Tool	Input	Output
1	`t3000_haystack_list_tags`	`filter?: "haystack"|"brick"|"custom"`	`{ tags: [{ tag_name, category, doc, parents, source, usage_count }], total }`
2	`t3000_haystack_get_point_tags`	`serial_numbers: int[], point_type?: "INPUT"|"OUTPUT"|"VARIABLE"`	`[{ serial_number, point_type, point_index, point_id, tag_name }]`
3	`t3000_haystack_search_points`	`tags: string[], serial_numbers?: int[], point_types?: string[]`	`{ entries: [{ point_id, serial_number, point_type, point_index, tag_name }] }`
4	`t3000_haystack_auto_tag`	`serial_numbers: int[]`	`{ tagged_count: int }`
5	`t3000_haystack_preview_tags`	`serial_numbers: int[]`	`{ preview: [{ point_id, tags: [], brick_class }] }`
6	`t3000_haystack_list_rules`	—	`{ rules: [{ id, rule_name, category, pattern, haystack_tags, enabled }] }`
7	`t3000_haystack_get_brick_class`	`serial_numbers: int[]`	`[{ serial_number, point_type, point_index, brick_class }]`

3.2 Core — 3 tools (new)

#	Tool	Input	Output
8	`t3000_ping`	—	`{ status, timestamp, server }`
9	`t3000_get_version`	—	`{ name, version, protocolVersion, toolCount }`
10	`t3000_describe_tool`	`tool_name: string`	`{ name, description, inputSchema }`

3.3 Data & Metadata — 5 tools (new)

#	Tool	Input	Output
11	`t3000_device_list`	`filter_name?: string`, `refresh?: boolean`	`[{ serial, name, device_type, input_count, output_count, variable_count, online, last_checked, refreshed }]`
12	`t3000_device_get_points`	`serial_number: int`, `point_type?: "INPUT"|"OUTPUT"|"VARIABLE"`	`[{ point_type, point_index, label, engineering_units, haystack_tags, brick_class }]`
13	`t3000_point_get_metadata`	`serial_number: int`, `point_type: string`, `point_index: int`	`{ serial_number, point_type, point_index, label, engineering_units, range_low, range_high, description, haystack_tags, brick_class, digital_analog }`
14	`t3000_metadata_search`	`query: string`, `serial_numbers?: int[]`, `point_types?: string[]`, `limit?: int`	`[{ serial_number, point_type, point_index, label, haystack_tags, brick_class }]`
15	`t3000_point_search`	`query: string`, `serial_numbers?: int[]`, `point_types?: string[]`, `limit?: int`	`[{ serial_number, point_type, point_index, label, haystack_tags, brick_class }]`

3.4 Operational — 5 tools (new)

#	Tool	Input	Output
16	`t3000_point_read`	`serial_number: int`, `point_type: string`, `point_index: int`	`{ serial_number, point_type, point_index, label, value, engineering_units, timestamp }`
17	`t3000_point_write`	`serial_number: int`, `point_type: string`, `point_index: int`, `value: number|boolean`, `confirm: boolean`	`{ success, written_value, timestamp }`
18	`t3000_point_read_batch`	`points: [{ serial_number, point_type, point_index }]`	`[{ serial_number, point_type, point_index, label, value, engineering_units, timestamp }]`
19	`t3000_point_write_batch`	`points: [{ serial_number, point_type, point_index, value }]`, `confirm: boolean`	`{ success, count }`
20	`t3000_point_batch_metadata`	`points: [{ serial_number, point_type, point_index }]`	`[{ serial_number, point_type, point_index, label, engineering_units, haystack_tags, brick_class }]`

3.5 Analytics — 2 tools (new)

#	Tool	Input	Output
21	`t3000_haystack_validate`	`serial_numbers?: int[]`	`{ passed, warnings: [{ point_id, issue }], errors: [{ point_id, issue }] }`
22	`t3000_haystack_export`	`serial_numbers: int[]`, `format: "haystack-json"|"brick-ttl"|"brick-jsonld"`	Format-specific model dump

3.6 Rules Management — 2 tools (new)

#	Tool	Input	Output
23	`t3000_rule_toggle`	`rule_id: int`, `enabled: boolean`	`{ rule_id, rule_name, enabled }`
24	`t3000_rule_create`	`pattern: string`, `haystack_tags: string[]`, `category: "haystack"|"brick"`, `priority?: int`, `brick_class?: string`, `units?: string`, `object_types?: string`	`{ rule_id, rule_name, pattern, category, haystack_tags, enabled }`

3.7 Alarms & Trends — 3 tools (new)

#	Tool	Input	Output
25	`t3000_alarm_list`	`serial_numbers?: int[]`, `active_only?: boolean`	`[{ alarm_id, serial_number, name, severity, message, timestamp, acknowledged, acknowledged_at }]`
26	`t3000_alarm_acknowledge`	`serial_number: int`, `alarm_id: int`	`{ success, acknowledged_at }`
27	`t3000_trendlog_query`	`serial_number: int`, `point_type: string`, `point_index: int`, `start: ISO8601`, `end?: ISO8601`, `limit?: int`	`{ serial_number, point_type, point_index, label, engineering_units, data: [{ timestamp, value }] }`

3.8 Device Operations — 4 tools (new)

#	Tool	Input	Output
28	`t3000_trendlog_list`	`serial_number: int`	`[{ log_id, point_type, point_index, label, interval }]`
29	`t3000_trendlog_export`	`serial_number: int`, `format?: string`	`{ filename, data, format }`
30	`t3000_device_refresh`	`serial_number: int`	`{ success, message }`
31	`t3000_schedule_list`	`serial_number: int`	`[{ schedule_id, name, days, periods }]`

3.9 Settings — 3 tools (new)

#	Tool	Input	Output
32	`t3000_settings_read`	`serial_number: int`	`{ settings: { name, baud_rate, ... } }`
33	`t3000_settings_write`	`serial_number: int`, `settings: object`, `confirm: boolean`	`{ success }`
34	`t3000_device_control`	`serial_number: int`, `command: string`, `params?: object`	`{ success, message }`

3.10 Control Logic — 5 tools (new)

#	Tool	Input	Output
35	`t3000_program_list`	`serial_number: int`	`[{ program_id, name, size }]`
36	`t3000_program_read`	`serial_number: int`, `program_id: string`	`{ program_id, name, source }`
37	`t3000_alarm_settings_read`	`serial_number: int`	`[{ alarm_setting_id, point, condition, thresholds }]`
38	`t3000_users_list`	`serial_number: int`	`[{ user_id, name, access_level }]`
39	`t3000_graphics_list`	`serial_number: int`	`[{ graphic_id, label, picture_file }]`

3.11 Documentation — 5 tools (new)

#	Tool	Input	Output
40	`t3000_doc_list`	`section?: string`	`[{ path, title, section }]`
41	`t3000_doc_read`	`path: string`	`{ path, title, content }`
42	`t3000_pid_list`	`serial_number: int`	`[{ pid_id, name, setpoint, kp, ki, kd }]`
43	`t3000_holiday_list`	`serial_number: int`	`[{ holiday_id, name, start_date, end_date }]`
44	`t3000_building_summary`	`serial_numbers?: int[]`	`{ devices, points, tags, alarms }`

4. Naming Convention

Prefix	Scope	Count
(none)	Server-level	3
`haystack_`	Ontology: tags, Brick, auto-tagging, validation, export	9
`device_`	Device inventory, points, refresh, control	5
`point_`	Live values: read, write, batch, metadata	5
`rule_`	Auto-tagging rule CRUD	2
`metadata_`	Cross-point search	1
`alarm_`	Alarm query, acknowledge, settings	3
`trendlog_`	Historical data, listing, export	3
`schedule_`	Time-based schedules	1
`settings_`	Device configuration	2
`program_`	PLC program CRUD	2
`users_`	User management	1
`graphics_`	Visualization screens	1
`doc_`	Documentation articles	2
`pid_`	PID loop listing	1
`holiday_`	Holiday schedules	1
`building_`	System overview	1

5. Protocol

MCP Streamable HTTP (2025-03-26) over `/api/mcp`. No bridge needed.

5.1 Transport

Method	Endpoint	Purpose
`POST`	`/api/mcp`	JSON-RPC 2.0 requests (client → server)
`GET`	`/api/mcp`	SSE endpoint (server → client notifications)
`DELETE`	`/api/mcp`	Session termination

Session management via `Mcp-Session-Id` header — returned in every POST response, validated on GET/DELETE.

5.2 JSON-RPC Methods

Standard methods: `initialize`, `t3000_ping`, `tools/list`, `tools/call`. Notifications: `notifications/initialized` (accepted, no response).

```
TEXT
```

```
Request:
{ "jsonrpc": "2.0", "id": 1, "method": "tools/call",
  "params": { "name": "<tool>", "arguments": { ... } } }

Response (success):
{ "jsonrpc": "2.0", "id": 1,
  "result": { "content": [{ "type": "text", "text": "<json>" }] } }

Response (tool error — isError: true):
{ "jsonrpc": "2.0", "id": 1,
  "result": { "content": [{ "type": "text", "text": "<error>" }], "isError": true } }

Response (protocol error):
{ "jsonrpc": "2.0", "id": 1,
  "error": { "code": -32602, "message": "..." } }
```

5.3 MCP Compliance

Spec Requirement	Status
Streamable HTTP transport (POST + GET + DELETE)	✅
`Mcp-Session-Id` header	✅
`initialize` with `protocolVersion`, `serverInfo`, `capabilities`	✅ `2025-03-26`
`capabilities.tools.listChanged: true`	✅
`notifications/initialized` (no response)	✅
Protocol-level `t3000_ping`	✅
`tools/list` with `name`, `title`, `description`, `inputSchema`	✅
`tools/call` success: `{ content: [{ type: "text", text }] }`	✅
`tools/call` error: `{ content: [...], isError: true }`	✅
Standard JSON-RPC error codes	✅
Works with all modern MCP clients (VS Code, Claude, Cursor, etc.)	✅

6. Safety: Write Confirmation

`t3000_point_write` and `t3000_point_write_batch` reject writes to OUTPUT/VARIABLE points unless `confirm: true` is set. INPUT points are exempt.

`confirm`	OUTPUT/VARIABLE	INPUT
`false`	❌ Error	✅ Accepted
`true`	✅ Accepted	✅ Accepted

7. Validation Rules (`t3000_haystack_validate`)

Rule	Condition	Severity
`sensor` → INPUT	Point tagged `sensor` must be INPUT type	error
`cmd` → OUTPUT	Point tagged `cmd` must be OUTPUT type	error
`air` disambiguation	Tag `air` present without `temp` / `humidity` / `pressure` / `flow` / `quality`	warning
Brick type consistency	Brick class implies sensor → must be INPUT; implies actuator → must be OUTPUT	error
Orphaned tags	Tag in `HAYSTACK_POINT_TAGS` not in `HAYSTACK_TAGS`	warning

8. Export Formats (`t3000_haystack_export`)

Format	MIME	Structure
`haystack-json`	`application/json`	`[{ id, dis, tags: { ... } }]` — Project Haystack v4
`brick-ttl`	`text/turtle`	`@prefix brick: <https://brickschema.org/schema/Brick#> .` — OWL/RDF
`brick-jsonld`	`application/ld+json`	`{ "@context": {...}, "@graph": [...] }` — JSON-LD

9. Implementation

9.1 Module: `api/src/haystack/mcp.rs`

Section	Change
`TOOLS` lazy_static	All 44 `ToolDef` entries with `name`, `title`, `description`, `inputSchema`
`handle_request` match	4 methods: `initialize`, `t3000_ping`, `tools/list`, `tools/call`
`mcp_post_handler`	Session management via `Mcp-Session-Id`, notification handling
`mcp_sse_handler`	GET /api/mcp SSE endpoint
`mcp_delete_handler`	DELETE /api/mcp session termination
`handle_tools_call`	`isError: true` for tool errors (LLM-friendly)
`execute_tool`	44 tool handlers + input validation

9.2 Service Dependencies

Tool	Calls
`t3000_ping`, `t3000_get_version`, `t3000_describe_tool`	Static / `TOOLS` array
`t3000_device_list`	`T3DeviceService`
`t3000_device_get_points`, `t3000_point_get_metadata`	Point tables + `tags_service` + Brick class query
`t3000_metadata_search`	`tags_service::search_points`
`t3000_point_read`, `t3000_point_read_batch`	Raw SQL SELECT fValue
`t3000_point_write`, `t3000_point_write_batch`	Raw SQL UPDATE fValue
`t3000_haystack_validate`	`HAYSTACK_POINT_TAGS` + `HAYSTACK_POINT_BRICK_CLASS` + point tables
`t3000_haystack_export`	All tagged points → serialize
`t3000_rule_toggle`, `t3000_rule_create`	`auto_tagging_service`
`t3000_alarm_list`, `t3000_alarm_acknowledge`	Alarm tables
`t3000_trendlog_query`	`T3TrendlogDataService`

9.3 Files Not Touched (unchanged from v4.0)

- `api/src/server.rs` — no route changes (MCP already registered)
- `api/src/t3_device/` — no changes (existing services used as-is)
- `api/src/haystack/tags_routes.rs` — no changes
- `api/src/haystack/auto_tagging_routes.rs` — no changes
- `api/src/haystack/tags_service.rs` — no changes
- `api/src/haystack/auto_tagging_service.rs` — no changes
- All frontend files — no changes
- All database migrations — no changes

9.4 Removed (v4.1)

- `tools/t3000-mcp-bridge.js` — no longer needed; server speaks Streamable HTTP natively

---

10. Changelog

v4.1 — 2026-07-20 (MCP Streamable HTTP compliance)

Change	Detail
Transport	POST + GET(SSE) + DELETE `/api/mcp` (Streamable HTTP)
Sessions	UUID sessions via `Mcp-Session-Id` header
Protocol version	`2025-03-26`
Capabilities	`tools.listChanged: true`
`notifications/initialized`	Accepted as JSON-RPC notification (no response)
Protocol `t3000_ping`	Added separate from tool `t3000_ping`
Tool errors	`isError: true` in result (not JSON-RPC error)
Tool `title`	All 44 tools have `title` field
`t3000_describe_tool`	Includes `title` in output
`t3000_get_version`	Uses `PROTOCOL_VERSION` constant
`t3000_point_get_metadata`	`point_type` validated before SQL (security)
Bridge	Removed `tools/t3000-mcp-bridge.js` — no longer needed

v4.0 — 2026-07-20

Change	Detail
Tools	Extended from 7 to 44 across 11 categories
Categories	Haystack, Core, Data, Operational, Analytics, Rules, Alarms
File changes	`mcp.rs` only

11. Database Tables (unchanged)

Table	Rows (approx)
`HAYSTACK_TAGS`	719
`HAYSTACK_TAG_RELATIONS`	~800
`HAYSTACK_POINT_TAGS`	533
`HAYSTACK_POINT_BRICK_CLASS`	~533
`HAYSTACK_AUTO_TAGGING_RULES`	228
`INPUTS` / `OUTPUTS` / `VARIABLES`	~192 per device
`ALARMS`	device-dependent
`TRENDLOG_DATA`	device-dependent

Version 4.0 — 2026-07-20

Haystack v1 — Legacy Single-Table Design

> ⚠ **DEPRECATED — June 2026** Replaced by v2-haystack-current-implementation.md

Purpose

This guide explains exactly how to integrate Project Haystack 4 for T3000 input points, what must be done manually one time, what is automatic, and what data is generated for each point.

Scope of this guide:

- API support and file download support only
- No outbound push workflow
- Input points first (from INPUTS table)

What Haystack Is In This Project

Haystack is a data modeling standard. In this project, integration means:

- Add semantic tags for points such as point, sensor, temp, kind
- Expose standardized APIs to read tagged metadata and history
- Provide file download in Haystack compatible formats

Important:

- You do not manually tag every point row forever
- You define mapping rules once, then code applies them automatically

Existing Source Tables Used

This project already has source tables in webview_t3_device schema:

- DEVICES
- INPUTS
- OUTPUTS
- VARIABLES

For this guide, only INPUTS is used for the end-to-end example.

New Table To Add

Create one new table for normalized Haystack entities.

SQL

```sql
CREATE TABLE IF NOT EXISTS HAYSTACK_ENTITY (
    id TEXT PRIMARY KEY,              -- example: dev1001.in0
    kind TEXT NOT NULL,               -- site | equip | point
    dis TEXT,                         -- display name
    tags TEXT NOT NULL,               -- JSON object as string
    serial_number INTEGER,            -- link to DEVICES.SerialNumber
    point_table TEXT,                 -- INPUTS | OUTPUTS | VARIABLES
    point_index TEXT,                 -- Input_Index / Output_Index / Variable_Index
    updated_at INTEGER                -- epoch millis
);

CREATE INDEX IF NOT EXISTS idx_haystack_entity_kind ON HAYSTACK_ENTITY(kind);
CREATE INDEX IF NOT EXISTS idx_haystack_entity_serial ON HAYSTACK_ENTITY(serial_number);
CREATE INDEX IF NOT EXISTS idx_haystack_entity_point_table ON HAYSTACK_ENTITY(point_tabl
```

Manual Work Versus Automatic Work

Manual one-time work:

1. Add migration to create HAYSTACK_ENTITY
2. Implement mapping function from INPUTS row to Haystack tags
3. Implement seeder or upsert job that runs on startup and after sync
4. Add API endpoints and file download endpoint

Automatic recurring work:

1. Read INPUTS rows
2. Compute tags by rule
3. Insert or update HAYSTACK_ENTITY rows
4. Recompute tags on future updates

No manual per-point editing is required in normal operation.

One Input Point End-To-End Example

Source INPUTS row:

- SerialNumber: 1001
- Input_Index: 0
- Label: Zone Temp
- Full_Label: Office 201 Zone Temperature

- fValue: 72.3
- Units: F
- Digital_Analog: 1

Generated Haystack entity:

- id: dev1001.in0
- kind: point
- dis: Office 201 Zone Temperature
- serial_number: 1001
- point_table: INPUTS
- point_index: 0
- tags JSON:

```JSON
{
  "point": "M",
  "sensor": "M",
  "temp": "M",
  "his": "M",
  "kind": "Number",
  "unit": "F",
  "curVal": 72.3,
  "equipRef": "dev1001"
}
```

Meaning:

- point M: this entity is a point
- sensor M: this point is a sensor
- temp M: this point is temperature related
- kind Number: value is numeric

Mapping Rules For INPUTS

Minimum mapping rules:

1. Base tags:

- point = M
- sensor = M

- his = M

2. Kind mapping from Digital_Analog:

- Digital_Analog = 1 -> kind = Number
- Digital_Analog = 0 -> kind = Bool

3. Semantic mapping from Units:

- F or C -> temp = M
- % -> humidity = M
- ppm -> co2 = M
- V -> volt = M and elec = M
- A -> current = M and elec = M

4. Value mapping:

- unit = Units
- curVal = parsed numeric value from fValue

5. Relationship mapping:

- equipRef = dev + SerialNumber

Recommended Seeder Behavior

Run seeder at two times:

1. Backend startup
2. After each successful device or point sync

Upsert strategy:

- If id does not exist in HAYSTACK_ENTITY, insert row
- If id exists, update dis, tags, updated_at

This ensures tag changes are automatic if metadata changes.

Rust Pseudocode For INPUTS Upsert

RUST

```rust
for input in load_all_inputs(db).await? {
    let id = format!("dev{}.in{}", input.serial_number, input.input_index);
    let dis = pick_display_name(input.full_label.as_deref(), input.label.as_deref());

    let mut tags = serde_json::Map::new();
    tags.insert("point".into(), serde_json::json!("M"));
    tags.insert("sensor".into(), serde_json::json!("M"));
    tags.insert("his".into(), serde_json::json!("M"));

    if input.digital_analog == "1" {
        tags.insert("kind".into(), serde_json::json!("Number"));
    } else {
        tags.insert("kind".into(), serde_json::json!("Bool"));
    }

    match input.units.as_deref().unwrap_or("") {
        "F" | "C" => { tags.insert("temp".into(), serde_json::json!("M")); }
        "%" => { tags.insert("humidity".into(), serde_json::json!("M")); }
        "ppm" => { tags.insert("co2".into(), serde_json::json!("M")); }
        "V" => {
            tags.insert("volt".into(), serde_json::json!("M"));
            tags.insert("elec".into(), serde_json::json!("M"));
        }
        "A" => {
            tags.insert("current".into(), serde_json::json!("M"));
            tags.insert("elec".into(), serde_json::json!("M"));
        }
        _ => {}
    }

    if let Some(unit) = input.units.as_deref() {
        tags.insert("unit".into(), serde_json::json!(unit));
    }
    if let Ok(v) = input.fvalue.parse::<f64>() {
        tags.insert("curVal".into(), serde_json::json!(v));
    }

    tags.insert("equipRef".into(), serde_json::json!(format!("dev{}", input.serial_numbe
```

```
    upsert_haystack_entity(
        db,
        &id,
        "point",
        &dis,
        &serde_json::Value::Object(tags).to_string(),
        input.serial_number,
        "INPUTS",
        &input.input_index,
    ).await?;
}
```

Required API Endpoints

Implement only these endpoints for current scope.

1. POST /api/haystack/about

- Returns server metadata and supported Haystack version

2. POST /api/haystack/read

- Filter entities by tags
- Returns Haystack grid style response

3. POST /api/haystack/hisRead

- Input: entity id and time range
- Reads trend history from existing trendlog data
- Returns ts and val rows in Haystack grid style

4. GET /api/haystack/download

- Query parameters: format, filter, range, ids
- Streams file attachment
- Recommended formats: json and csv initially

Suggested Download Behavior

For MVP:

- format=json: download Haystack grid JSON
- format=csv: download flattened rows

Response headers:

- Content-Type based on format
- Content-Disposition: attachment; filename="haystack_export_."

Validation Checklist

1. Migration applied and HAYSTACK_ENTITY exists
2. Seeder inserted rows for INPUTS points
3. One known point returns expected tags (point, sensor, kind)
4. If Digital_Analog changes, kind updates on next upsert
5. read endpoint returns filtered point
6. hisRead endpoint returns time series for that point
7. download endpoint saves file successfully

Common Mistakes To Avoid

1. Manual per-point tagging in UI

- Do not do this except special override cases

2. Storing tags in many separate columns

- Keep tags as JSON for flexibility

3. One-time seed only

- Must also run upsert after sync to keep data current

4. Hardcoding only one unit format

- Support both plain and symbol variants where possible

Next Increment After INPUTS

After INPUTS is stable, repeat the same pattern for:

- OUTPUTS
- VARIABLES

Use id patterns:

- dev{SerialNumber}.out{Output_Index}
- dev{SerialNumber}.var{Variable_Index}

Then expose them through the same read and download endpoints.

mcp-claude-desktop
Claude Desktop MCP Setup

> ⬅️ Back to MCP Server tab

Connect Claude Desktop to the T3000 MCP server to let Claude query devices, read/write points, manage Haystack tags, and run analytics.

Covers: **Claude Desktop**, **Cursor**, **Cline**, **Continue.dev** — all use the same `mcp-remote` approach.

Prerequisites

- T3000 API running on port `9103`
- Node.js installed (provides `npx`)
- Claude Desktop installed

Step 1: Open Claude Settings

Open Claude Desktop → **Settings** → **Developer** → **Edit Config**

Step 2: Paste the Config

Replace `<host>` with `localhost` (local) or the machine's LAN IP (remote):

JSON

```
{
  "mcpServers": {
    "T3000": {
      "command": "npx",
      "args": [
        "-y",
        "mcp-remote",
        "http://<host>:9103/api/mcp",
        "--allow-http"
      ]
    }
  }
}
```

Example for local:

JSON

```
{
  "mcpServers": {
    "T3000": {
      "command": "npx",
      "args": [
        "-y",
        "mcp-remote",
        "http://localhost:9103/api/mcp",
        "--allow-http"
      ]
    }
  }
}
```





Step 3: Restart Claude Desktop

Close and reopen Claude Desktop. On first run, `npx` will automatically download `mcp-remote`.

Step 4: Verify Connection

Look for the 🔌 **plug icon** in Claude's interface — it confirms the MCP server is connected.

Step 5: Try a Query

Ask Claude any of these:

Query	Expected Tool
List the Haystack auto-tagging rules	`t3000_haystack_list_rules`
Show me the input points for device T3-NB-ESP	`t3000_device_get_points`

See the full MCP API examples for all 44 tools with prompt text.

Other Clients

Client	Config Location	Config Format
Cursor	`.cursor/mcp.json`	Same as Claude Desktop
Cline	MCP Servers view in VS Code	Same as Claude Desktop
Continue.dev	`config.json`	Same as Claude Desktop

Troubleshooting

Issue	Fix
`npx` not found	Install Node.js from nodejs.org
Connection refused	Ensure T3000 API is running on port 9103
Tools not appearing	Check Claude logs: `Help` → `View Logs`
44 tools expected	Verify you see all 11 categories: Haystack, Core, Data, Operational, Analytics, Rules, Alarms, Device Ops, Settings, Control Logic, Documentation

---

Available Tools (44)

Category	Tools
Haystack	`t3000_haystack_list_tags`, `t3000_haystack_get_point_tags`, `t3000_haystack_search_points`, `t3000_haystack_auto_tag`, `t3000_haystack_preview_tags`, `t3000_haystack_list_rules`, `t3000_haystack_get_brick_class`
Core	`t3000_ping`, `t3000_get_version`, `t3000_describe_tool`
Data & Metadata	`t3000_device_list`, `t3000_device_get_points`, `t3000_point_get_metadata`, `t3000_metadata_search`, `t3000_point_search`
Operational	`t3000_point_read`, `t3000_point_write`, `t3000_point_read_batch`, `t3000_point_write_batch`, `t3000_point_batch_metadata`
Analytics	`t3000_haystack_validate`, `t3000_haystack_export`
Rules	`t3000_rule_toggle`, `t3000_rule_create`
Alarms & Trends	`t3000_alarm_list`, `t3000_alarm_acknowledge`, `t3000_trendlog_query`
Device Operations	`t3000_trendlog_list`, `t3000_trendlog_export`, `t3000_device_refresh`, `t3000_schedule_list`
Settings	`t3000_settings_read`, `t3000_settings_write`, `t3000_device_control`
Control Logic	`t3000_program_list`, `t3000_program_read`, `t3000_alarm_settings_read`, `t3000_users_list`, `t3000_graphics_list`
Documentation	`t3000_doc_list`, `t3000_doc_read`, `t3000_pid_list`, `t3000_holiday_list`, `t3000_building_summary`





mcp-vscode-copilot
VS Code Copilot MCP Setup

> ⬅️ Back to MCP Server tab

Connect VS Code Copilot to the T3000 MCP server to let Copilot query devices, read/write points, and manage Haystack tags.

Prerequisites

- T3000 API running on port `9103`
- GitHub Copilot extension installed in VS Code (includes built-in MCP support)

Step 1: Create MCP Config

In your project root, create `.vscode/mcp.json`. For global access across all projects, place it at `C:\Users\<username>\AppData\Roaming\Code\User\mcp.json` (Windows) or `~/.config/Code/User/mcp.json` (macOS/Linux).

JSON

```
{
  "servers": {
    "T3000": {
      "type": "http",
      "url": "http://<host>:9103/api/mcp"
    }
  }
}
```

Replace `<host>` with `localhost` (local) or the machine's LAN IP (remote).

> **Note:** VS Code uses `"servers"` (not `"mcpServers"` like Claude Desktop).

Settings	Config

Step 2: Reload VS Code

Press `Ctrl+Shift+P` → type **Reload Window** → press Enter.

Step 3: Verify Connection

Press `Ctrl+Shift+P` → type **MCP: List Servers** → press Enter.

Confirm the `T3000` server shows as **connected**. If it shows an error or "disconnected", check that:

- The T3000 API is running on port 9103
- The URL in `mcp.json` is reachable (ping the host)





Step 4: Try a Query

Open Copilot Chat and test any tool:

Ask Copilot	Expected Tool
List the Haystack auto-tagging rules	`t3000_haystack_list_rules`
Show me the input points for device T3-NB-ESP	`t3000_device_get_points`

See the full MCP API examples for all 44 tools with prompt text.

Troubleshooting

Issue	Fix
44 tools not appearing	Reload VS Code window (`Ctrl+Shift+P` → Reload Window)
Connection refused	Ensure T3000 API is running on port 9103
Config not picked up	Verify file is at `.vscode/mcp.json` (not `mcpServers`)
Tools list but calls fail	Check T3000 API logs for errors

Available Tools (44)

Category	Tools
Haystack	`t3000_haystack_list_tags`, `t3000_haystack_get_point_tags`, `t3000_haystack_search_points`, `t3000_haystack_auto_tag`, `t3000_haystack_preview_tags`, `t3000_haystack_list_rules`, `t3000_haystack_get_brick_class`
Core	`t3000_ping`, `t3000_get_version`, `t3000_describe_tool`
Data & Metadata	`t3000_device_list`, `t3000_device_get_points`, `t3000_point_get_metadata`, `t3000_metadata_search`, `t3000_point_search`
Operational	`t3000_point_read`, `t3000_point_write`, `t3000_point_read_batch`, `t3000_point_write_batch`, `t3000_point_batch_metadata`
Analytics	`t3000_haystack_validate`, `t3000_haystack_export`
Rules	`t3000_rule_toggle`, `t3000_rule_create`
Alarms & Trends	`t3000_alarm_list`, `t3000_alarm_acknowledge`, `t3000_trendlog_query`
Device Operations	`t3000_trendlog_list`, `t3000_trendlog_export`, `t3000_device_refresh`, `t3000_schedule_list`
Settings	`t3000_settings_read`, `t3000_settings_write`, `t3000_device_control`
Control Logic	`t3000_program_list`, `t3000_program_read`, `t3000_alarm_settings_read`, `t3000_users_list`, `t3000_graphics_list`
Documentation	`t3000_doc_list`, `t3000_doc_read`, `t3000_pid_list`, `t3000_holiday_list`, `t3000_building_summary`

MCP API Examples

> ⬅️ Back to MCP Server tab | Setup: VS Code Copilot | Setup: Claude Desktop

Complete reference of natural-language prompts for 50+ MCP tools across 16 categories. Copy any prompt and paste into Copilot Chat or Claude — the LLM automatically maps your question to the right tool and parameters.

Haystack Tagging 7 tools

`t3000_haystack_list_tags` — Discover available tags

Discover the full Haystack tag vocabulary. Tags are organized by category: `haystack` (standard v4 tags like `air`, `sensor`, `temp`), `brick` (Brick ontology classes), and `custom` (user-defined). Returns tag names, descriptions, parent relationships, and how many points use each tag.

What Haystack tags are available?

Show me all Haystack tags in the brick category

List custom tags defined in the system

`t3000_haystack_get_point_tags` — Get tags assigned to points

Retrieve all Haystack tags assigned to specific device points. Filter by device serial number and optionally by point type (INPUT, OUTPUT, VARIABLE). Returns a flat list of point→tag assignments showing what semantic meaning each point carries.

What tags are assigned to device 233626?

Get tags for the INPUT points on device 237219

Show me all tags for OUTPUT points on devices 233626 and 240488

`t3000_haystack_search_points` — Find points by tags

Search across all devices for points that have specific combinations of Haystack tags. Points must match ALL specified tags. Use this to find all temperature sensors, all outside air points, all command points,

etc. Optionally restrict by device serials or point types.

Search for all temperature sensors

Find outside air temperature sensors

Show me all humidity sensors on device 237219

`t3000_haystack_auto_tag` — Run auto-tagging

Apply the auto-tagging engine to devices. First applies range-based rules (digital/analog, engineering units, value ranges), then regex rules that match point labels. Assigns Haystack tags AND Brick ontology classes to every point. Returns the count of points tagged.

Auto-tag device 233626

Auto-tag all my devices: 233626, 237219, 240488

`t3000_haystack_preview_tags` — Preview tags without saving

See exactly what tags and Brick classes would be assigned by auto-tagging BEFORE writing to the database. Essential for testing new rules or verifying tag assignments. Returns full preview with tags, Brick class, and the rule that matched for each point.

Preview what tags would be assigned to device 240488

Show me a preview of auto-tagging for device 233626 without applying

`t3000_haystack_list_rules` — Show all tagging rules

Lists every auto-tagging rule in the system with its regex pattern, target tags, Brick class (if any), category, priority, and enabled/disabled status. Currently 248 rules covering Haystack v4 tags and Brick ontology classes.

List the Haystack auto-tagging rules

Show me all enabled tagging rules in the brick category

`t3000_haystack_get_brick_class` — Get Brick ontology classes

Brick is a formal ontology for buildings (brickschema.org). After auto-tagging, each point gets a Brick class like `Supply_Air_Temperature_Sensor` or `Outside_Air_Flow_Sensor`. Use this to check what

Brick classes are assigned to your device points.

What Brick class does input 8 on device 237219 have?

Show all Brick classes for device 233626

Data & Discovery 7 tools

`t3000_device_list` — Enumerate all devices

Get every T3000 device in the system with serial number, name, device type, building/floor/room, and counts of INPUT, OUTPUT, and VARIABLE points. Optionally filter by device name substring or refresh the device list to discover new devices on the network.

List all T3000 devices

Find devices named T3-NB-ESP

How many devices are in this system?

Refresh device list to discover new devices

`t3000_device_get` — Get a single device record

Query one device by serial number and return its full record: serial, name, product type, IP address, building/floor/room, online/offline status, panel and network info, and timestamps. Use this when you need a specific device's details rather than the whole list.

Get the full details for device 233626

What is the IP address of device 240488?

Is device 237219 online?

`t3000_scan_network` — Scan network for T3000 devices

Scan the local network for T3000 devices via UDP broadcast (0x64/0x65 protocol). Returns discovered devices with serial numbers, IP addresses, product types, firmware versions, panel names, and subnet info. Takes 5–10 seconds. Use this to discover new or replacement devices on the network.

Scan the network for T3000 devices

Discover any new T3000 panels on the network

Find all T3000 devices on my LAN

`t3000_device_get_points` — Get all points for a device

Returns every point on a device — labels, engineering units, Haystack tags, and Brick classes. Optionally filter by point type to get only INPUTs, OUTPUTs, or VARIABLEs.

Show me the input points for device T3-NB-ESP

Show all VARIABLE points on device 233626

List all points on device 240488

`t3000_point_get_metadata` — Full metadata for one point

Deep-dive into a single point: label, full label, engineering units, range (low/high), description, digital vs analog, all Haystack tags, and Brick class. Useful when you need to understand exactly what a point represents.

Get full metadata for input 0 on device 240488

What are the units and range for output 3 on device 237219?

`t3000_metadata_search` — Cross-device label search

Search point labels across all devices by keyword. Matches against point labels, full labels, and descriptions. Optionally filter by device serials and point types. Great for finding all points related to "temperature", "flow", "pressure", etc.

Search for points labeled temperature

Search for flow across all output points

Find all points with fan in the name on device 233626

`t3000_point_search` — Semantic point search

Search points across devices using natural language. Matches against point labels, Haystack tags, Brick classes, and descriptions. Returns ranked results by relevance. Much smarter than label search —

"supply air temperature" matches `Supply_Air_Temperature_Sensor` even if the label is abbreviated.

Find the temperature sensor in the lobby

Show me all fan speed outputs

Search for discharge air temperature on device 233626

Operational — Read/Write 5 tools

`t3000_point_read` — Read a single point value

Read the current (last synced) value of any point from the database. Returns the value, engineering units, label, and timestamp of the last reading.

Read input point 0 on device 233626

What's the current value of output 3 on device 237219?

Read variable 12 on device 240488

`t3000_point_write` — Write to any point field

Write to any writable field on a point. Defaults to `value` (fValue). Also supports: `label`, `description`, `range`, `auto_manual`, `digital_analog`. All other fields are preserved from the current device state. **Safety:** requires `confirm: true` for OUTPUT and VARIABLE points. Writes go through the C++ FFI layer to the actual device.

Set output 5 on device 233626 to 72.5

Turn on output 7 on device 237219

Rename input 2 on device 240488 to "ZoneTemp"

Change description of output 3 to "Main Fan Speed"

Set the range of variable 0 on device 233626 to 100

`t3000_point_read_batch` — Read multiple points at once

Read values for multiple points in a single call. Points can span different devices and point types. Much faster than calling `t3000_point_read` repeatedly for bulk operations.

Read inputs 0, 1, and 2 on device 240488 all at once

Read input 0 on 233626 and output 0 on 237219 together

`t3000_point_write_batch` — Write multiple points at once

Write values to multiple points in a single call. Each point can specify an optional `field` (defaults to `value`). Failures are reported per-point; partial success is supported. Requires `confirm: true`. Points can span different devices.

Set outputs 0 through 3 on device 237219 to 100

Set output 1 to 80 and rename it to "PumpSpeed"

`t3000_point_batch_metadata` — Get full metadata for multiple points

Get complete metadata for multiple points in a single call. Returns label, engineering units, range, digital/analog type, description, current value, Haystack tags, and Brick class for each point. Much more efficient than calling `t3000_point_get_metadata` N times — use when you need full context on several points at once.

Give me full metadata for inputs 0 through 4 on device 240488

What are the tags, units, and ranges for output 0 of device 233626 and input 1 of device 237219?

Device Operations 8 tools

`t3000_trendlog_list` — Discover available trendlogs

List all trendlogs configured for a device. Returns trendlog IDs, labels, logging interval, buffer size, and how many points each trendlog tracks. Use this to discover what historical data is available before querying with `t3000_trendlog_query`.

What trendlogs are available on device

List all logging configurations for device

240488?

233626

`t3000_trendlog_export` — Export trendlog history as CSV/JSON

Export all historical data from a trendlog in one call. Queries every point in the trendlog and returns timestamped values. Use after `t3000_trendlog_list` to discover trendlog IDs. Defaults to CSV format with columns: timestamp, point_type, point_index, point_id, value, units, range, digital_analog.

Export trendlog 1 from device 240488 as CSV for the last 24 hours

Export all trendlog data from device 233626 trendlog 0 as JSON since 2026-01-01

`t3000_device_refresh` — Force refresh from hardware

Refresh point data directly from the physical device via the C++ FFI layer (Action 17). Updates the database with the latest values from the hardware. Optionally filter by point type to refresh only inputs, outputs, or variables.

Refresh all data from device 240488

Refresh only the inputs on device 233626

`t3000_schedule_list` — List device schedules

List all schedules configured on a device. Returns schedule IDs, daily time settings for each day of the week (Monday–Friday), assigned outputs/variables, holiday settings, and interval configuration.

Show me the schedules on device 240488

What time-based automation is configured on device 233626?

`t3000_settings_read` — Read device settings

Read all device configuration from the database. Returns 8 categories: network (IP/subnet/gateway/DHCP), communication (COM ports/baudrates/parity/stopbits), time (timezone/NTP/DST), protocol (Modbus ID/MSTP/BACnet), DynDNS, hardware info, feature flags, and email settings. Optionally filter to a single category.

Read all settings for device 240488

Show me the network configuration of device 233626

What are the communication settings for device 240488?

`t3000_settings_write` — Update device settings

Update device configuration fields. Supports network (ip_address, subnet, gateway, tcp_type), communication (com0/1/2_config, com_baudrate0/1/2), time (time_zone, enable_sntp, sntp_server), and email (smtp_server, email_address, etc.). Requires confirm:true.

Set device 240488 to use DHCP

Change the NTP server on device 233626 to pool.ntp.org

Configure COM1 on device 240488 as Modbus Slave at 19200 baud

`t3000_device_control` — Reboot or factory reset

Send control commands to a device: reboot (restart the controller) or reset_defaults (factory reset to default settings). Requires confirm:true for safety.

Reboot device 240488

Factory reset device 233626

`t3000_device_delete` — Delete a device record

Remove a device and its database record by serial number. Destructive and irreversible — requires confirm:true. Only the local T3000 database record is removed; the physical controller on the network is unaffected. Use `t3000_device_list` first to confirm the serial number before deleting.

Delete device 233626 from the project

Remove device 271845 from the database

Control Logic 5 tools

`t3000_program_list` — List PLC programs

List all control logic programs running on a device. Returns program IDs, labels, status (running/stopped), auto/manual mode, program size, and switch node. Use before

`t3000_program_read` to find which programs exist.

List all programs on device 240488

What control logic is running on device 233626?

`t3000_program_read` — Read program source code

Read a specific PLC program's full details including source code (truncated to 2000 chars), label, status, auto/manual mode, size, and switch node. The full source length is included in the response.

Show me the source code of program 1 on device 240488

What does program 'MAIN_LOOP' on device 233626 do?

`t3000_pid_list` — List PID control loops

List all PID control loops on a device. Returns loop IDs, current setpoint, process variable (input value), output value, P/I/D tuning parameters (proportional, reset, rate), bias, action type, auto/manual mode, setpoint limits, and status. Essential for HVAC diagnostics.

List all PID loops on device 240488

Show me the tuning parameters for the PID loops on device 233626

Which PID loops are in manual mode?

`t3000_holiday_list` — List holiday exceptions

List all holiday schedule exceptions on a device. Returns holiday IDs, dates (month/day/year), holiday output values, auto/manual mode, and status. Holidays override the normal weekly schedule on their designated dates.

List holiday schedules on device 240488

What holiday exceptions are configured on device 233626?

`t3000_building_summary` — System-wide overview

Get a one-shot dashboard of the entire building automation system. Returns total device count with names, active alarm count, total trendlogs, schedules, programs, and PID loops across all devices.

Includes a health indicator (good/warning/critical based on alarm count). Perfect for "How's the building?" queries.

How is the building doing right now?

Give me a system overview

Are there any active alarms in the system?

Diagnostics 2 tools

`t3000_device_diagnostics` — Single device health check

Run a comprehensive diagnostic check on a single device. Returns: device name, firmware version, hardware revision, IP address, point counts (inputs/outputs/variables), active alarm count, trendlog count, program status (total/running), schedule count, PID loop status (total/in-auto), and an overall health rating (good/warning/needs_attention).

Run diagnostics on device 233626

Check the health of device 240488

`t3000_device_diagnostics_batch` — Multi-device health check

Run diagnostics on multiple devices or all devices at once. Returns per-device health reports plus an overall building health summary (good/warning/needs_attention). Pass no serial_numbers to diagnose everything. Use for system-wide health assessments.

Check the health of all devices

Run diagnostics on devices 233626, 237219, and 240488

Config & Users 3 tools

`t3000_alarm_settings_read` — Read alarm thresholds

Read alarm threshold configuration for a device. Returns alarm rules: which points are monitored, the comparison condition, low/high/normal/way-low/way-high threshold values, and time delays. This is alarm configuration — use `t3000_alarm_list` for active alarms.

Read alarm threshold settings for device 240488

What alarm rules are configured on device 233626?

`t3000_users_list` — List device users

List all users configured on a device. Returns user IDs, names, access levels (View/Full/Graphic/Routine), rights, default panel/group assignments, and status.

List all users on device 240488

Who has access to device 233626?

`t3000_graphics_list` — List graphic screens

List all graphic/HMI screens available on a device. Returns graphic IDs, labels, descriptions, picture files, total points per screen, and switch nodes.

List graphic screens on device 240488

What HMI screens are available on device 233626?

Documentation 2 tools

`t3000_doc_list` — List documentation topics

List all T3000 documentation topics organized by section: Quick Start, Architecture, Device Management, Data Points, Features, API Reference, Guides, Building Platform, and Haystack & MCP. Use to discover what docs exist before reading a specific one.

What documentation is available?

Show me the API reference docs

`t3000_doc_read` — Read a documentation page

Read the full markdown content of a T3000 documentation page. Pass the path from `t3000_doc_list` (e.g. `quick-start/overview` or `haystack/mcp-api-examples`). Fetches from local filesystem in dev mode, falls back to GitHub raw in production.

Show me the MCP API examples documentation

Read the quick start guide

What does the troubleshooting guide say about connection issues?

Core 3 tools

`t3000_ping` — Health check

Simple connectivity test. Returns server status, current timestamp, and server name.

Is the T3000 MCP server running?

Ping the MCP server

`t3000_get_version` — Server metadata

Returns server name, version number, MCP protocol version (2025-03-26), and total tool count.

What version is the MCP server?

`t3000_describe_tool` — Tool documentation

Get the complete input schema, description, and parameter details for any tool by name. Useful for LLM agents to understand tool capabilities before calling.

Describe the device_list tool

What parameters does trendlog_query accept?

Analytics & Export 2 tools

`t3000_haystack_validate` — Validate tagging quality

Run ontology validation rules against tagged points. Checks for: sensor tags on non-INPUT points, command tags on non-OUTPUT points, missing required tags, conflicting tag combinations, invalid Brick class assignments, and orphaned tag references. Returns warnings and errors.

Validate the Haystack tags on device 237219

Validate tagging across all devices

Are there any tagging errors in device 233626?

`t3000_haystack_export` — Export semantic model

Export the full semantic model for devices in standard formats. **haystack-json**: Project Haystack tagged entity format. **brick-ttl**: Brick ontology in Turtle RDF (W3C standard). **brick-jsonld**: Brick ontology in JSON-LD (linked data). **csv-flat**: Flat CSV table of all points with serial_number, point_type, point_index, label, description, current value, units, range, digital/analog, Haystack tags, and Brick class. Use for integration with other building systems or semantic analysis tools.

Export device 233626 as Brick Turtle RDF

Export all three devices as Haystack JSON

Export device 240488 as Brick JSON-LD

Export device 233626 as a flat CSV spreadsheet

Rules Management 2 tools

`t3000_rule_toggle` — Enable or disable rules

Toggle an auto-tagging rule on or off by its rule ID. Disabled rules are skipped during auto-tagging but remain in the database. Use `t3000_haystack_list_rules` to find rule IDs.

Disable auto-tagging rule 5

Enable rule 3

`t3000_rule_create` — Create a new tagging rule

Add a custom auto-tagging rule. Specify a regex pattern to match point labels, the Haystack tags to assign, and optionally a Brick class. Rules can target specific point types, engineering units, or object types. Higher priority rules are applied first.

Create a rule that tags any point with CO2 in the label as air, co2, sensor

Add a Brick rule to classify ZoneTemp labels as Zone_Air_Temperature_Sensor

---

Alarms & Trends 3 tools

`t3000_alarm_list` — List alarms

Get all alarms in the system with severity, message, timestamp, and acknowledgment status. Filter by device serials or show only unacknowledged (active) alarms.

List all active alarms

Show me alarms for device 233626

Are there any unacknowledged alarms on device 237219?

`t3000_alarm_acknowledge` — Acknowledge an alarm

Acknowledge a specific alarm by device serial number and alarm ID. Acknowledged alarms are marked with a timestamp. Use `t3000_alarm_list` to find alarm IDs.

Acknowledge alarm abc123 on device 233626

`t3000_trendlog_query` — Query historical trend data

Retrieve time-series data for any point over a specified time range. Specify start and optional end time in ISO 8601 format, and limit the number of data points returned. Returns timestamp-value pairs with engineering units and point metadata.

Get trend data for input 8 on device 237219 for the last hour

Show me trend data for variable 0 on device 240488 from yesterday to now

Get the last 100 readings for output 2 on device 233626

Task Management 4 tools

`t3000_task_create` — Create a workflow task

Create a new task for tracking commissioning, maintenance, or troubleshooting workflows. Tasks have a title, description, status (pending/in_progress/completed), priority (low/normal/high/critical), and optional device reference.

Create a task: configure AHU-1 network settings

Add a high-priority task to troubleshoot device 233626 alarms

`t3000_task_list` — List tasks

List all tasks with optional filters by status (pending, in_progress, completed) or device serial number. Returns tasks sorted by creation time, newest first.

Show me all pending tasks

What tasks are associated with device 240488?

`t3000_task_update` — Update a task

Update a task's status, title, description, or priority. Use to mark tasks as in_progress or completed as you work through a workflow.

Mark task "Configure AHU-1" as completed

Change priority of task "Calibrate sensors" to high

`t3000_task_delete` — Delete a task

Delete a task by ID. Use to clean up completed or obsolete tasks.

Delete the completed commissioning tasks

Site Memory 3 tools

`t3000_memory_save` — Save site knowledge

Save a note about the building site for future reference. Memories persist across sessions and are automatically loaded into the AI context. Use for site-specific conventions, device naming patterns, or user preferences.

Remember that AHU-3 is the main unit and AHU-2 is decommissioned

Save a note: device naming convention is Building-Floor-Type-Number

`t3000_memory_list` — List site memories

List all saved site memories with optional filtering by category or text search. Returns memories sorted by last update time.

What do we know about this site?

Show me memories about AHU layout

`t3000_memory_delete` — Delete a memory

Delete a specific memory entry by key. Use to remove outdated or incorrect site information.

Forget the old AHU layout notes

Navigation 6 tools

`t3000_nav_list` — List all UI pages

List all pages in the T3000 web UI with paths, titles, keyboard shortcuts, and whether they require a device to be selected. Use to help users find the right page.

What pages are available in the T3000?

List all monitoring pages

`t3000_nav_search` — Search UI pages

Search for T3000 pages and topics by keyword. Returns matching pages ranked by relevance. Use when the user asks "How do I..." or "Where is..." questions about the T3000 interface.

Where do I configure PID loops?	**How do I set up schedules?**

`t3000_nav_redirect` — Navigate to a page

Get the URL to navigate to a specific T3000 page, optionally with a device pre-selected. The frontend uses this URL to redirect the user.

Open the outputs page for device 233626	**Take me to the alarms page**

`t3000_page_info` — Page details

Get detailed information about a T3000 page: what it does, what you can view/edit/configure, related MCP tools, keyboard shortcuts, and available features.

What can I do on the Alarms page?	**Tell me about the PID Loops page**

`t3000_device_current` — Current device context

Get the currently selected device in the web UI. Returns device type, name, and a note about its role. Use to discover which device the user is working with before querying points.

Which device am I currently working with?	**What device is selected right now?**

`t3000_set_chat_device` — Set the device to use for MCP operations

Confirm which device the AI should use for MCP operations. Call after the user confirms which device to work with (which may differ from the UI-selected device). This sets the authoritative chat device context.

Use device 233626 for the rest of this conversation	**Switch the current device to 240488**

Quick Test

After setup, try these two to confirm everything works:

List the Haystack auto-tagging rules

`haystack_list_rules`

Show me the input points for device T3-NB-ESP

`device_get_points`

AI Integration — Local LLM Server

⬅️ Back to Docs | Cloud LLM APIs | AI Chat Page Design

How to run a local LLM server on your network and connect T3000 to it. No cloud API keys needed — all processing stays inside your building.

Architecture

TEXT



1. Browser sends user message to `POST /api/ai/chat`
2. T3000 forwards to your local LLM via OpenAI-compatible `/v1/chat/completions`
3. LLM streams response — text goes straight to browser via SSE
4. When LLM wants to call a tool, T3000 intercepts the `tool_use` event
5. Tool is executed **locally** by the MCP handler — never sent to the LLM
6. Tool result is sent back to the LLM, which continues the stream

No building data leaves your network. No cloud API keys needed.

Option 1: Ollama (easiest)

Install

Windows / Mac / Linux: Download from ollama.com

Or on Linux:

```shell
curl -fsSL https://ollama.com/install.sh | sh
```

Pull a Model

```shell
## Best for tool calling
ollama pull llama3.1:8b
ollama pull qwen2.5:7b
ollama pull mistral:7b

## Smaller/faster
ollama pull phi3:mini
ollama pull gemma2:2b
```

Configure for Tool Calling

Ollama needs a Modelfile to support OpenAI-compatible tool calling:

```shell
ollama pull llama3.1:8b

## Create a custom modelfile (optional but helps with structured outputs)
cat > Modelfile << 'EOF'
FROM llama3.1:8b
PARAMETER temperature 0.1
PARAMETER num_ctx 8192
SYSTEM You are a building automation assistant. You have access to tools that query devi
EOF

ollama create t3k-assistant -f Modelfile
```

Verify It's Running

```shell
ollama serve
## or just run:
ollama list
```

Default port: `11434`

T3000 Settings

Field	Value
Provider	Local
Model	`llama3.1:8b` or your model name
Endpoint	`http://localhost:11434/v1`
API Key	(leave blank — Ollama has no auth by default)

If T3000 is on a different machine, use the server's LAN IP:

```text
http://192.168.1.50:11434/v1
```

Tool Calling Support

Model	Tool calling	Notes
`llama3.1:8b`	✅ Good	Best overall for tool use
`qwen2.5:7b`	✅ Good	Strong at structured outputs
`mistral:7b`	⚠️ Partial	May miss tools occasionally
`phi3:mini`	❌ Limited	Not recommended for tool chains

Option 2: vLLM (production-grade)

Install

SHELL

```
pip install vllm
```

Start Server with Tool-Ready Model

SHELL

```
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-7B-Instruct \
  --port 8000 \
  --served-model-name qwen2.5 \
  --enable-auto-tool-choice \
  --tool-call-parser hermes
```

For GPU:

SHELL

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-8B-Instruct \
  --port 8000 \
  --tensor-parallel-size 2 \
  --enable-auto-tool-choice \
  --tool-call-parser llama3_json
```

T3000 Settings

Field	Value
Provider	Local
Model	`Llama-3.1-8B-Instruct`
Endpoint	`http://localhost:8000/v1`
API Key	`token vllm` (default vLLM auth)

Hardware Requirements

Model Size	VRAM	RAM (CPU-only)
7B (qwen2.5)	8 GB	16 GB
8B (llama3.1)	10 GB	20 GB
70B (llama3.1)	40 GB × 2	Not recommended

Option 3: LM Studio (GUI, easiest setup)

Install

1. Download from lmstudio.ai
2. Install and open the app
3. Search for `Llama 3.1 8B Instruct` or `Qwen 2.5 7B Instruct`
4. Download the model
5. Go to **Developer** tab → **Start Server**
6. Set port to `1234`

T3000 Settings

Field	Value
Provider	Local
Model	`llama-3.1-8b-instruct`
Endpoint	`http://localhost:1234/v1`
API Key	(leave blank)

Option 4: llama.cpp (lightweight, no GPU needed)

Install

```shell
## Clone and build
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
make -j4

## Download a GGUF model (example: Qwen 2.5 7B Q4)
wget https://huggingface.co/bartowski/Qwen2.5-7B-Instruct-GGUF/resolve/main/Qwen2.5-7B-I
```

Start Server

```shell
./llama-server \
  -m Qwen2.5-7B-Instruct-Q4_K_M.gguf \
  --host 0.0.0.0 \
  --port 8080 \
  -c 8192
```

T3000 Settings

Field	Value
Provider	Local
Model	`qwen2.5-7b`
Endpoint	`http://localhost:8080/v1`
API Key	(leave blank)

Network Security for Local Servers

If T3000 and LLM are on the SAME machine

```text
Endpoint: http://localhost:{port}/v1
```

No network exposure — safest option.

If T3000 and LLM are on DIFFERENT machines (same LAN)

```text
Endpoint: http://192.168.1.50:{port}/v1
```

For Ollama, you must also allow LAN access:

```shell
## On the Ollama server machine:
set OLLAMA_HOST=0.0.0.0
ollama serve
```

Firewall: Ensure port `11434` (Ollama), `8000` (vLLM), or your custom port is open on the LLM machine's firewall.

Do NOT expose to the internet

Local LLM servers have no authentication by default. Only use on your internal network behind a firewall. For remote access, use a VPN.

API Format (OpenAI-Compatible)

All local servers use the same OpenAI-compatible chat completions API:

```text
POST http://{host}:{port}/v1/chat/completions
```

Request:

JSON

```
{
  "model": "llama3.1:8b",
  "messages": [
    {"role": "system", "content": "You are a building automation assistant..."},
    {"role": "user", "content": "What's AHU-1 supply temperature?"}
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "t3000_haystack_search_points",
        "description": "Search points by Haystack tags.",
        "parameters": {
          "type": "object",
          "properties": {
            "tags": {"type": "array", "items": {"type": "string"}},
            "serial_numbers": {"type": "array", "items": {"type": "integer"}}
          },
          "required": ["tags"]
        }
      }
    },
    {
      "type": "function",
      "function": {
        "name": "t3000_point_read",
        "description": "Read live value of a point.",
        "parameters": {
          "type": "object",
          "properties": {
            "serial_number": {"type": "integer"},
            "point_type": {"type": "string", "enum": ["INPUT", "OUTPUT", "VARIABLE"]},
            "point_index": {"type": "integer"}
          },
          "required": ["serial_number", "point_type", "point_index"]
        }
      }
    }
  ],
```

```
  "stream": true
}
```

Streaming response: SSE chunks with `data: {"choices":[{"delta":{"content":"The "}}]}` format.

Tool call response:

JSON

```
{
  "choices": [
    {
      "delta": {
        "tool_calls": [
          {
            "index": 0,
            "id": "call_abc123",
            "function": {
              "name": "t3000_haystack_search_points",
              "arguments": "{\"tags\":[\"supply\",\"air\",\"temp\"]}"
            }
          }
        ]
      }
    }
  ]
}
```

Tool result format (send back):

JSON

```
{
  "role": "tool",
  "tool_call_id": "call_abc123",
  "content": "[{\"label\":\"AHU-1 Supply Temp\",\"value\":55.2}]"
}
```

Recommended Local Models

Model	Size	Tool Calling	Notes
`llama3.1:8b`	8B	✅ Excellent	Best all-around for tool use
`qwen2.5:7b`	7B	✅ Excellent	Strong at JSON/formatted output
`mistral-nemo:12b`	12B	✅ Good	Larger context window
`gemma2:9b`	9B	⚠️ Good	May need prompt tuning
`phi3:mini`	3.8B	❌ Weak	Too small for reliable tool calls

Troubleshooting

"Model does not support tools"

Your model is too old or doesn't support function calling. Switch to `llama3.1:8b` or `qwen2.5:7b`.

"Connection refused"

The LLM server isn't running. Check:

```shell
curl http://localhost:11434/api/tags      # OLlama
curl http://localhost:8000/v1/models       # vLLM
```

"Tool call returned empty"

The model didn't understand the prompt. Try:

- More explicit tool descriptions
- Adding examples in the system prompt
- Using a larger model (8B+ instead of 3B)

"Response is slow"

- Check CPU/GPU usage on the LLM server
- Reduce `num_ctx` (context length) for faster inference
- Use a smaller model for simple queries, larger for complex ones

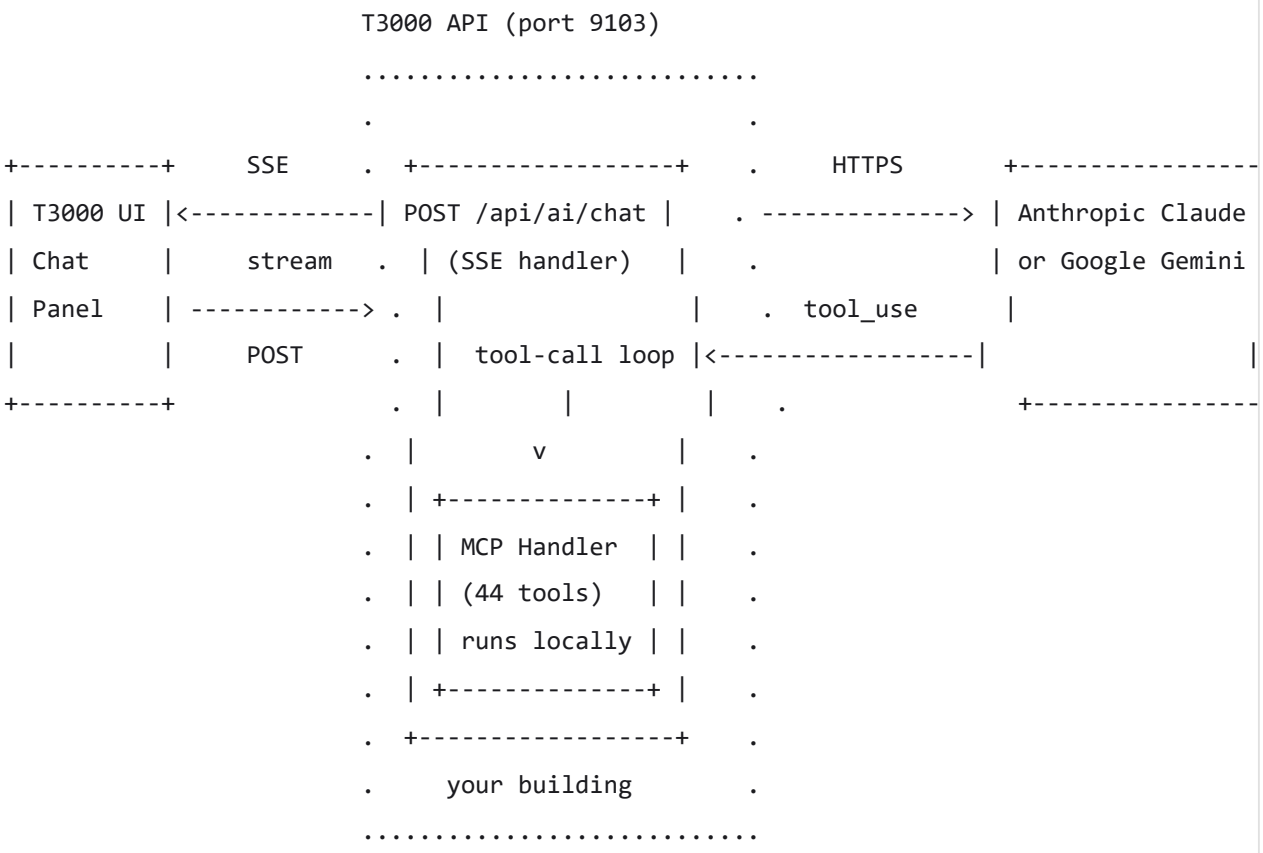
AI Integration — Cloud LLM APIs

⬅️ Back to Docs | Local LLM Server | AI Chat Page Design

How T3000 connects to cloud-hosted LLM providers (Anthropic Claude, Google Gemini) for the built-in AI chat panel. The T3000 backend acts as a secure proxy — your API key never leaves the server.

Architecture

TEXT



1. Browser sends user message to `POST /api/ai/chat`
2. T3000 forwards to the cloud LLM (Claude or Gemini) over HTTPS
3. LLM streams response — text chunks go straight to browser via SSE
4. When LLM wants to call a tool, T3000 intercepts the `tool_use` event
5. Tool is executed **locally** by the MCP handler — **no building data sent to the cloud provider**
6. Tool result is sent back to the LLM, which continues the stream

The browser talks only to T3000 via SSE. T3000 forwards to the LLM, intercepts tool calls, executes them against the MCP handler locally, and streams the text response back.

Provider 1: Anthropic (Claude)

API Endpoint

```
TEXT

POST https://api.anthropic.com/v1/messages
```

Headers

Header	Value	Required
`x-api-key`	`sk-ant-api03-...`	Yes
`anthropic-version`	`2023-06-01`	Yes
`Content-Type`	`application/json`	Yes

Request (with streaming & tools)

JSON

```
{
  "model": "claude-3-5-sonnet-20241022",
  "max_tokens": 4096,
  "system": "You are a building automation assistant for the T3000 platform...",
  "messages": [
    {"role": "user", "content": "What's AHU-1 supply temperature?"}
  ],
  "tools": [
    {
      "name": "t3000_haystack_search_points",
      "description": "Search points by Haystack tags.",
      "input_schema": {
        "type": "object",
        "properties": {
          "tags": {"type": "array", "items": {"type": "string"}, "description": "Tags to
          "serial_numbers": {"type": "array", "items": {"type": "integer"}, "description
        },
        "required": ["tags"]
      }
    },
    {
      "name": "t3000_point_read",
      "description": "Read live value of a single point.",
      "input_schema": {
        "type": "object",
        "properties": {
          "serial_number": {"type": "integer"},
          "point_type": {"type": "string", "enum": ["INPUT", "OUTPUT", "VARIABLE"]},
          "point_index": {"type": "integer"}
        },
        "required": ["serial_number", "point_type", "point_index"]
      }
    }
  ],
  "stream": true
}
```

Streaming Response (SSE events)

The response is a series of Server-Sent Events:

Event	Meaning
`message_start`	Conversation begins, includes `message.id`
`content_block_start`	A new block starts — `text` or `tool_use`
`content_block_delta`	Incremental text chunk (`text_delta`) or tool args (`input_json_delta`)
`content_block_stop`	Current block completed
`message_delta`	Stop reason (`end_turn` or `tool_use`)
`message_stop`	Full turn complete

Tool Call Cycle

When Claude decides to use a tool, you'll see:

```TEXT
event: content_block_start
data: {"type":"content_block_start","index":0,"content_block":{"type":"tool_use","id":"t

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":{"type":"input_json_delta","partia
```

After executing the tool locally, send back:

```JSON
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01Axyz...",
      "content": "[{\"label\":\"AHU-1 Supply Temp\",\"value\":55.2,\"units\":\"Deg.C\",\
    }
  ]
}
```

Then continue the conversation — Claude will use the result in its next response.

Recommended Models

Model	Use case	Cost
`claude-3-5-sonnet-20241022`	General queries, dashboard Q&A	$$
`claude-3-haiku-20240307`	Fast lookups, status checks	$
`claude-3-opus-20240229`	Complex diagnostics, multi-AHU analysis	$$$

Get an API Key

1. Go to console.anthropic.com
2. Create an account or sign in
3. Navigate to **API Keys** → **Create Key**
4. Copy the key (starts with `sk-ant-api03-`)
5. Paste into T3000 → AI Settings

Provider 2: Google Gemini

API Endpoint

```
TEXT

POST https://generativelanguage.googleapis.com/v1beta/models/{MODEL}:streamGenerateConte
```

The API key goes in the URL as a query parameter — not a header.

Headers

Header	Value
`Content-Type`	`application/json`

Request (with streaming & function calling)

JSON

```
{
  "system_instruction": {
    "parts": [{"text": "You are a building automation assistant for the T3000 platform..
  },
  "contents": [
    {
      "role": "user",
      "parts": [{"text": "What's AHU-1 supply temperature?"}]
    }
  ],
  "tools": [
    {
      "function_declarations": [
        {
          "name": "t3000_haystack_search_points",
          "description": "Search points by Haystack tags.",
          "parameters": {
            "type": "object",
            "properties": {
              "tags": {"type": "array", "items": {"type": "string"}},
              "serial_numbers": {"type": "array", "items": {"type": "integer"}}
            },
            "required": ["tags"]
          }
        },
        {
          "name": "t3000_point_read",
          "description": "Read live value of a single point.",
          "parameters": {
            "type": "object",
            "properties": {
              "serial_number": {"type": "integer"},
              "point_type": {"type": "string", "enum": ["INPUT", "OUTPUT", "VARIABLE"]},
              "point_index": {"type": "integer"}
            },
            "required": ["serial_number", "point_type", "point_index"]
          }
        }
```

```
      ]
    }
  ]
}
```

Streaming Response

Each SSE data line contains a chunk like:

JSON

```
{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [{"text": "The supply "}]
      }
    }
  ]
}
```

When Gemini calls a function:

JSON

```
{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [
          {
            "functionCall": {
              "name": "t3000_haystack_search_points",
              "args": {"tags": ["supply", "air", "temp"]}
            }
          }
        ]
      }
    }
  ]
}
```

Tool Response Format

Send the result back as a follow-up content:

JSON


```
{
  "contents": [
    {"role": "user", "parts": [{"text": "What's AHU-1 supply temperature?"}]},
    {
      "role": "model",
      "parts": [{"functionCall": {"name": "t3000_haystack_search_points", "args": {"tags
    },
    {
      "role": "function",
      "parts": [
        {
          "functionResponse": {
            "name": "t3000_haystack_search_points",
            "response": {"points": [{"label": "AHU-1 Supply Temp", "value": 55.2}]}
          }
        }
      ]
    }
  ]
}
```


Recommended Models

Model	Use case
`gemini-2.0-flash`	Fast, great for structured data extraction
`gemini-1.5-pro`	Complex reasoning, multi-step tool chains

Get an API Key

1. Go to aistudio.google.com/apikey
2. Click **Create API Key**
3. Copy the key
4. Paste into T3000 → AI Settings

Tool Format Mapping

The 25 MCP tools must be converted to each provider's format:

MCP Tool Schema	Anthropic	Gemini	OpenAI/Local
`input_schema`	`input_schema` (same)	`parameters` (same)	`function.parameters`
Array container	Direct `tools[]`	`tools[].function_declarations[]`	`tools[].function`
`$ref` / `definitions`	Supported	Flatten manually	Supported

T3000's backend normalizes all three formats from a single canonical MCP tool definition stored in code.

Rate Limits & Costs

Provider	Free Tier	Pay-as-you-go
Anthropic	None	$3/M input tokens, $15/M output (Sonnet)
Gemini	1,500 requests/day (Flash)	$0.075/M input, $0.30/M output (Flash)

Tool calls count as output tokens (Anthropic) or are free (Gemini, separate billing).

Security Notes

- T3000 stores API keys encrypted at rest using AES-256-GCM
- The browser never sees the raw API key — it is injected server-side
- All LLM traffic goes through T3000's own HTTPS connection
- Tool execution happens locally — no building data leaves your network to the LLM provider

AI Chat Page — Integration Design

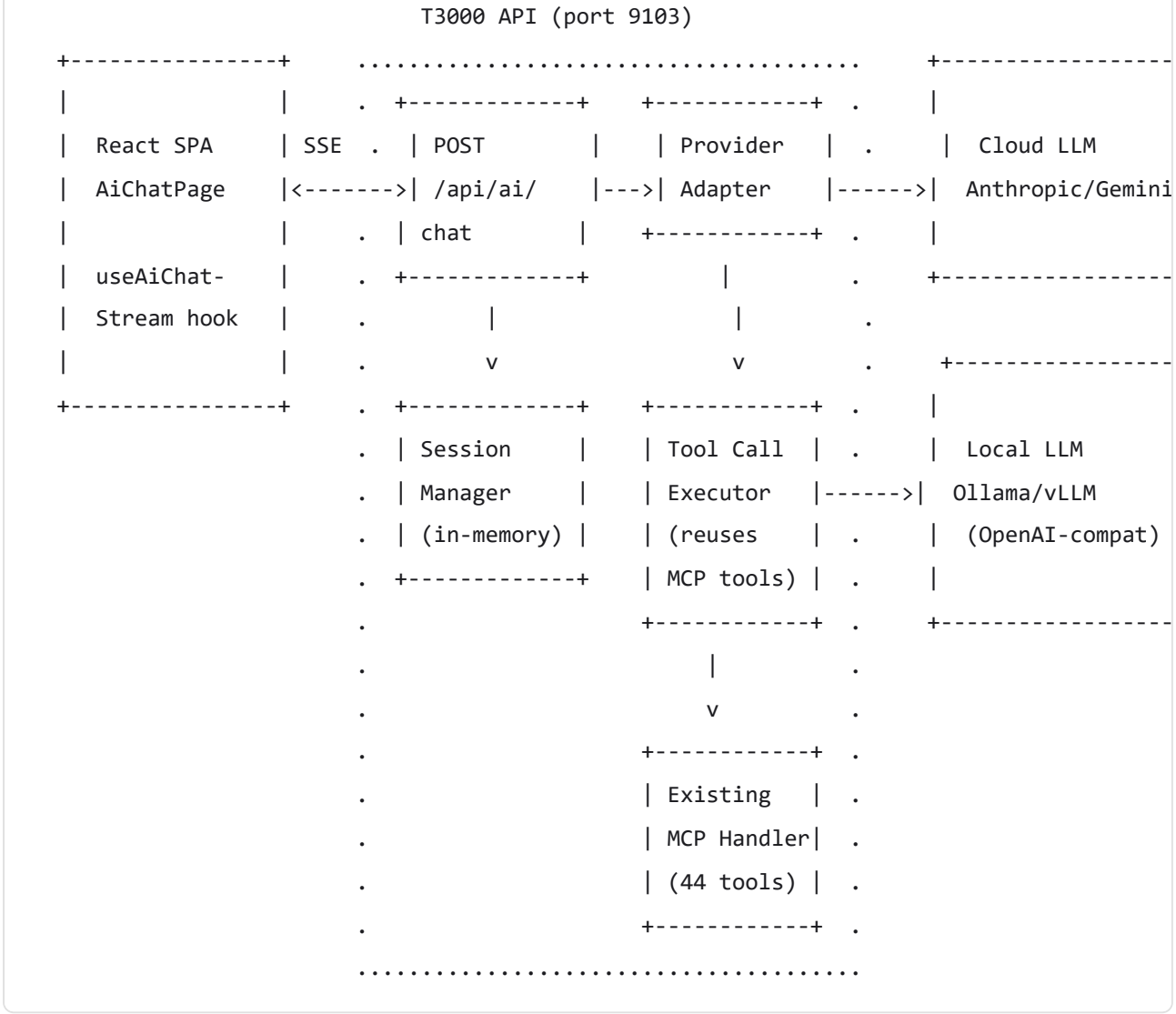
> Back to AI Docs | Local LLM Setup | Cloud LLM APIs

Complete integration plan for the built-in AI chat page. Covers backend SSE endpoint, LLM provider abstraction, tool-call loop, and React frontend.

---

1. Overall Architecture

TEXT



Key principle: The browser talks only to T3000 via SSE. T3000 proxies to the LLM, intercepts tool calls, executes them locally against the existing MCP machinery, and streams text back. No API keys ever reach the browser.

---

2. Backend Design

2.1 New Module: `api/src/ai/`

TEXT

```
api/src/ai/
  mod.rs              -- module root, re-exports create_ai_routes()
  types.rs            -- ChatRequest, Message, StreamEvent, AiSettings
  session.rs          -- in-memory session store (Arc<Mutex<HashMap>>)
  tool_executor.rs    -- maps tool names -> existing MCP handler functions
  routes.rs           -- axum route handlers (SSE streaming)
  providers/
    mod.rs            -- LlmProvider trait + dispatch
    local.rs          -- OpenAI-compatible (Ollama, vLLM, LM Studio, llama.cpp)
```

2.2 SSE Protocol

`POST /api/ai/chat` returns `text/event-stream`. Events sent from server to client:

TEXT

```
Event               Direction     Meaning
_____________________________________________________________________
text_delta          S -> C        Streaming text chunk, append to UI
tool_call           S -> C        LLM requested a tool, show pending badge
tool_result         S -> C        Tool executed locally, show summary
done                S -> C        Turn complete, contains session_id
error               S -> C        Fatal error, stop streaming
```

Request body (POST /api/ai/chat):

JSON

```
{
  "provider": "local",
  "model": "llama3.1:8b",
  "messages": [
    { "role": "user", "content": "What is AHU-1 supply temperature?" }
  ],
  "session_id": "optional-uuid-here",
  "settings": {
    "endpoint": "http://localhost:11434/v1",
    "api_key": ""
  }
}
```

SSE stream example (raw):

TEXT

```
data: {"event":"text_delta","content":"Let me"}

data: {"event":"text_delta","content":" check the"}

data: {"event":"tool_call","id":"call_01","name":"t3000_point_read","args":"{\"serial_nu

data: {"event":"tool_result","id":"call_01","result":"[{\"label\":\"AHU-1 Supply Temp\",

data: {"event":"text_delta","content":"The supply air"}

data: {"event":"text_delta","content":" temperature is 55.2"}

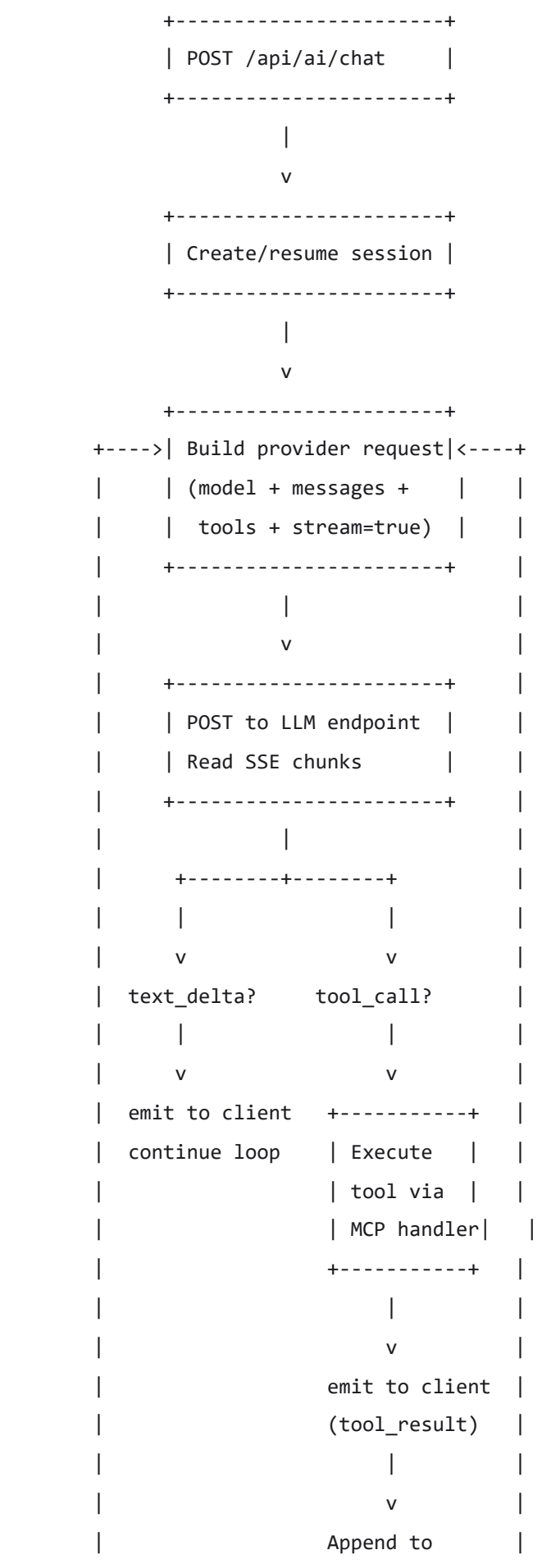
data: {"event":"text_delta","content":" Deg.C."}

data: {"event":"done","session_id":"abc123-def456"}
```

2.3 Tool-Call Loop (Route Handler)

The route handler manages the loop — NOT the provider. This keeps providers stateless and simple.

TEXT


POST /api/ai/chat
Create/resume session
Build provider request
(model + messages +
tools + stream=true)
POST to LLM endpoint
Read SSE chunks
text_delta?
tool_call?
emit to client
continue loop
Execute
tool via
MCP handler
emit to client
(tool_result)
Append to



2.4 Tool Executor Strategy

Two options — **Option A is recommended:**

Option	Approach	Pros	Cons
A: Direct	Call existing MCP handler functions directly	Zero overhead, typesafe	Requires `pub(crate)` refactor of `mcp.rs` dispatch
B: JSON-RPC	Construct synthetic `JsonRpcRequest`, call dispatch	No refactor needed	Double-serialization, error wrapping noise

Recommendation: Option A. Extract each `"tool_name" => { ... }` match arm in `mcp.rs` into a `pub(crate)` function. The tool executor calls these directly. This is a mechanical refactor — no logic changes.

2.5 Session Manager

TEXT

```
+-----------------------------------------------------+
| SessionManager (Arc<Mutex<HashMap<String,Session>>>)|
+-----------------------------------------------------+
| create(req: &ChatRequest) -> Session                |
| get(id: &str) -> Option<Session>                    |
| update_messages(id, messages)                       |
| delete(id)                                          |
| cleanup_expired(max_age: Duration)                   |
+-----------------------------------------------------+

Session {
    id: String,
    provider: String,
    model: String,
    endpoint: String,
    api_key: Option<String>,    // Never logged
    messages: Vec<Message>,     // Full history for tool-call loop
    created_at: Instant,
    last_active: Instant,
}
```

- Max 100 concurrent sessions
- Idle timeout: 1 hour (auto-purge via background tokio task every 5 min)
- Sessions survive server restart only if saved to DB (Phase 4 polish — not Phase 1)

2.6 Settings Storage

TEXT

```
+-------------------+       +------------------+       +------------------+
| AI Chat Page (UI) | <---- | GET /api/ai/     | <--- | settings table   |
|                   |       | settings         |       | (encrypted       |
|                   | ----> | PUT /api/ai/     | ---> |  api_key column) |
|                   |       | settings         |       +------------------+
+-------------------+       +------------------+

API key flow:
  1. User enters key in Settings panel
  2. Frontend POSTs to PUT /api/ai/settings
  3. Backend encrypts key with AES-256-GCM before storing
  4. Frontend receives { ok: true } (key masked to "sk-ant...****")
  5. On subsequent requests, frontend sends NO key — backend injects it
```

2.7 Provider Adapter Trait

RUST

```rust
// providers/mod.rs

#[async_trait]
pub trait LlmProvider: Send + Sync {
    /// Stream chat completion. The provider ONLY handles the HTTP request to
    /// the LLM and parses the SSE stream. Tool-call looping is done by the
    /// route handler — the provider just emits tool_call events.
    async fn stream_chat(
        &self,
        endpoint: &str,
        api_key: Option<&str>,
        model: &str,
        messages: &[Message],
        tools: &[ToolDef],          // Canonical tool definitions
        tx: Sender<StreamEvent>,    // Send events back to route handler
    ) -> Result<(), AiError>;
}
```

Three implementations:

- `LocalProvider` — OpenAI-compatible `/v1/chat/completions`

- `AnthropicProvider` — `/v1/messages` (Phase 3)
- `GeminiProvider` — `streamGenerateContent` (Phase 3)

2.8 Routes to Wire

In `server.rs` (after the MCP routes line):

```rust
.merge(crate::ai::create_ai_routes())
```

In `lib.rs`:

```rust
pub mod ai;   // Add after `pub mod haystack;`
```

2.9 Additional API Endpoints

```text
POST   /api/ai/chat           SSE streaming chat
DELETE /api/ai/sessions/:id   Clear a session
GET    /api/ai/settings       Get current provider/model/endpoint (key masked)
PUT    /api/ai/settings       Save provider settings (key encrypted)
GET    /api/ai/tools          List available tools (for debug/transparency)
```

---

3. Frontend Design

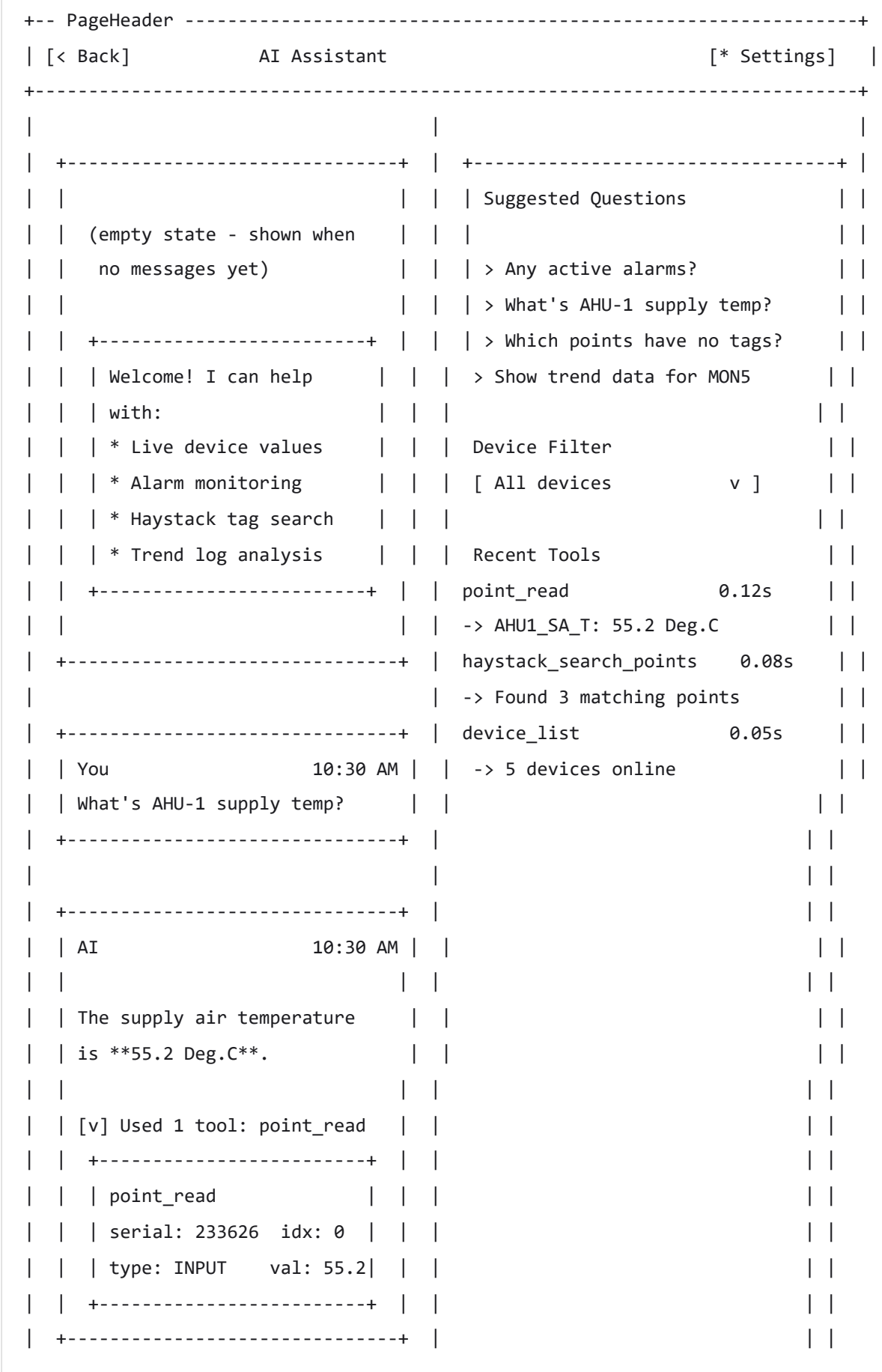
3.1 File Tree

TEXT

```
src/t3-react/features/ai-chat/
  index.ts                          -- barrel export
  pages/
    AiChatPage.tsx                   -- page component, layout
  hooks/
    useAiChatStream.ts               -- SSE client, message state machine
    useAiSettings.ts                 -- provider settings CRUD
  components/
    ChatPanel.tsx                    -- main chat area orchestrator
    ChatMessage.tsx                  -- single message bubble
    ChatInput.tsx                    -- input bar with send/stop
    SettingsDrawer.tsx               -- slide-over settings panel
    ContextPanel.tsx                 -- right sidebar (suggestions, device filter, tool
    EmptyState.tsx                   -- welcome screen with suggested questions
    ToolCallCard.tsx                 -- expandable tool call result inline
```

3.2 Page Layout (ASCII)

TEXT



```
|                                    |                                    | |
|  ... streaming indicator ...       |                                   | |
|  The supply fan is currently ▉     |                                   | |
|                                    |                                    | |
|  +-------------------------------+ |                                    | |
|  | Type a message...       [STOP] | |                                     | |
|  | local:llama3.1:8b  [v]         | |                                     | |
|  +-------------------------------+ |                                    | |
+------------------------------------+------------------------------------+
|  Collapse sidebar >                                                   <<  |
+-------------------------------------------------------------------------+
```

3.3 Component State Flow

TEXT

```
+----------+    +------------+    +------------------+    +---------+
| ChatInput |--->| ChatPanel  |--->| useAiChatStream  |--->| POST    |
| (user     |    | (orchestra-|    | (SSE reader,     |    | /api/ai/|
|  types    |    |  tor)      |    |  message store)  |    | chat    |
|  message) |    +------------+    +------------------+    +---------+
+----------+         |                    |
      ^              |                    |
      |              v                    v
      |       +------------+        +------------+
      +-------| ChatMessage|        | ToolCall   |
   (scroll    | (bubble    |        | Card       |
    back to   |  render)   |        | (expandable|
    input)    +------------+        +------------+
```

Message type in state:

TYPESCRIPT

```typescript
type ChatMessage =
  | { role: 'user';    content: string; timestamp: number }
  | { role: 'assistant'; content: string; toolCalls: ToolCallRecord[]; timestamp: number
  | { role: 'system';  content: string; timestamp: number };  // errors, info

type ToolCallRecord = {
  id: string;
  name: string;
  args: string;        // JSON
  result?: string;     // JSON, set when tool_result arrives
  status: 'pending' | 'success' | 'error';
};
```

3.4 SSE Client Hook (`useAiChatStream`)

TEXT

```
+-----------------------+
| useAiChatStream()     |
+-----------------------+
| messages: ChatMessage[]
| isStreaming: boolean
| streamingText: string         // partial text while streaming
| activeToolCalls: Map<id,Record>
| sessionId: string | null
+-----------------------+
| sendMessage(content): void
| abort(): void
| clearSession(): void
+-----------------------+

Internals:
  1. fetch('/api/ai/chat', { method:'POST', body: JSON })
  2. Get ReadableStream from response.body
  3. Pipe through TextDecoderStream
  4. Split on '\n\n', parse 'data: {...}' lines
  5. Dispatch by event type:
     text_delta  -> append to streamingText, update last assistant msg
     tool_call   -> insert ToolCallRecord with status:'pending'
     tool_result -> resolve ToolCallRecord, update status
     done        -> finalize message, store sessionId
     error       -> append system message with error text
  6. On stream end or abort: set isStreaming=false
```

3.5 Route & Toolbar Integration

Route: `/#/t3000/ai-chat` (under MainLayout, authenticated)

In `App.tsx` — add lazy import:

TYPESCRIPT

```typescript
const AiChatPage = React.lazy(() =>
  import('../features/ai-chat').then((m) => ({ default: m.AiChatPage }))
);
```

In `App.tsx` — add Route inside `<Route path="/t3000">`:

TYPESCRIPT

```tsx
<Route
  path="ai-chat"
  element={
    <React.Suspense fallback={<div>Loading...</div>}>
      <AiChatPage />
    </React.Suspense>
  }
/>
```

In `toolbarConfig.ts` — add button entry (before Array):

TYPESCRIPT

```ts
{
  id: 'toolbar-ai-chat',
  icon: ChatBubblesQuestionRegular,              // from @fluentui/react-icons
  label: 'AI Chat',
  tooltip: 'AI Assistant',
  action: 'openWindow',
  windowId: 16,
  route: '/t3000/ai-chat',
},
```

---

4. Complete Implementation Steps

Phase 1 — Core Plumbing (Local LLM, no tools)

TEXT

Step	File	Action
1.	api/src/ai/types.rs	CREATE — ChatRequest, Message, StreamEvent, AiSettings structs
2.	api/src/ai/session.rs	CREATE — SessionManager with create/get/delete/cleanup
3.	api/src/ai/providers/mod.rs	CREATE — LlmProvider trait
4.	api/src/ai/providers/local.rs	CREATE — LocalProvider (OpenAI compatible, SSE parsing)
5.	api/src/ai/routes.rs	CREATE — POST /api/ai/chat SSE handler, settings CRUD
6.	api/src/ai/mod.rs	CREATE — module root
7.	api/src/lib.rs	MODIFY — add `pub mod ai;`
8.	api/src/server.rs	MODIFY — merge ai routes
9.	src/.../ai-chat/hooks/useAiChatStream.ts	CREATE — SSE client hook
10.	src/.../ai-chat/hooks/useAiSettings.ts	CREATE — settings hook
11.	src/.../ai-chat/components/ChatMessage.tsx	CREATE — message bubble
12.	src/.../ai-chat/components/ChatInput.tsx	CREATE — input bar
13.	src/.../ai-chat/components/ChatPanel.tsx	CREATE — orchestrator
14.	src/.../ai-chat/components/EmptyState.tsx	CREATE — welcome screen
15.	src/.../ai-chat/pages/AiChatPage.tsx	CREATE — page layout
16.	src/.../ai-chat/index.ts	CREATE — barrel export
17.	src/t3-react/app/App.tsx	MODIFY — add lazy import + route
18.	src/t3-react/app/config/toolbarConfig.ts	MODIFY — add button

Phase 2 — Tool Calling

```
19.  api/src/ai/tool_executor.rs            CREATE — dispatch to MCP handlers
20.  api/src/haystack/mcp.rs                REFACTOR — extract handle_* fns
                                            as pub(crate) so tool_executor
                                            can call them directly
21.  src/.../ai-chat/components/ToolCallCard.tsx  CREATE — expandable card
22.  src/.../ai-chat/components/ContextPanel.tsx  CREATE — right sidebar
23.  api/src/ai/routes.rs                   MODIFY — add tool-call loop to
                                            SSE handler (the while loop)
```

Phase 3 — Cloud Providers + Settings

```
24.  api/src/ai/providers/anthropic.rs      CREATE — Anthropic adapter
25.  api/src/ai/providers/gemini.rs         CREATE — Gemini adapter
26.  src/.../ai-chat/components/SettingsDrawer.tsx CREATE — settings panel
```

Phase 4 — Polish

```
27.  Session persistence (save to DB, survive restart)
28.  Prompt template customization
29.  Suggested questions in empty state
30.  Device scoping (filter tools to one device)
31.  Mobile-responsive layout
```

---

5. Design Decisions & Rationale

5.1 Why SSE instead of WebSocket?

Feature	SSE	WebSocket
Direction	Server -> Client only (perfect for streaming)	Bidirectional
Reconnection	Built-in (EventSource API)	Manual
HTTP/2 multiplexing	Yes	No
Firewall friendly	Yes (plain HTTP)	Sometimes blocked
Complexity	Trivial	Moderate

SSE is the standard for LLM streaming. The frontend sends requests via `fetch POST` and receives responses via SSE. No WebSocket overhead needed.

5.2 Why in-memory sessions?

- Chat sessions are ephemeral — users don't expect history to survive a server restart
- Avoids database schema changes and migration complexity
- Simpler cleanup (just drop the map on restart)
- Max 100 sessions × ~50KB each = ~5MB memory — negligible
- Phase 4 can add DB persistence if needed

5.3 Why direct MCP handler calls (not JSON-RPC)?

The tool executor could construct a `JsonRpcRequest { method: "t3000_point_read", ... }` and call the existing MCP dispatch. But that means:

- Serialize args to JSON -> parse JSON -> dispatch -> serialize result -> parse result
- Two unnecessary (de)serialization passes

Instead, extract `pub(crate) async fn handle_point_read(args: Value, state: &T3AppState) -> Result<Value>` functions and call them directly. The MCP JSON-RPC endpoint still works — it just delegates to these same functions.

5.4 Why the route handler owns the tool-call loop?

Three options for tool-call loop placement:

Location	Pros	Cons
Provider	Self-contained	Provider must know about session, tool executor — violates SRP
Route handler	Clean separation, provider is stateless	Route handler is more complex
New "orchestrator" struct	Dedicated component	Over-engineering for v1

Route handler is the pragmatic choice. Providers become simple adapters that take messages + tools, return a stream of events. The route handler manages the session state and decides when to loop.

5.5 System Prompt Strategy

The system prompt is the most important factor for tool-calling quality. Default template (stored in settings, user-overridable):

```
TEXT
```

```
You are a building automation assistant for the T3000 platform.
You have access to tools that query real-time device data, search
Haystack semantic tags, read/write points, and check alarms.

Rules:
- ALWAYS use tools to fetch live data — never guess values.
- When asked about a device, use device_list first to find it.
- When asked about a point value, search by tags with
  haystack_search_points, then read with point_read.
- Keep responses concise. Include units when reporting values.
- If a tool fails, explain the error and suggest alternatives.
- For multi-point queries, use batch tools (point_read_batch).
```

6. Error Handling Matrix

TEXT

```
+------------------------------+----------------------------------------------+
| Scenario                     | User Experience                              |
+------------------------------+----------------------------------------------+
| LLM unreachable              | Red toast: "Cannot reach LLM server at       |
| (connection refused)         | localhost:11434. Is Ollama running?"          |
|                              | Input remains enabled for retry.             |
+------------------------------+----------------------------------------------+
| LLM returns non-200          | Toast with status + message. Input enabled.  |
+------------------------------+----------------------------------------------+
| Tool execution fails         | Tool result sent as error to LLM. LLM handles|
| (e.g., device offline)       | gracefully: "AHU-1 appears offline."         |
+------------------------------+----------------------------------------------+
| Session expired (>1h idle)   | Auto-creates new session. Subtle indicator:  |
|                              | "New session" badge appears briefly.         |
+------------------------------+----------------------------------------------+
| Network drops mid-stream     | "Connection lost" message. Retry button.     |
+------------------------------+----------------------------------------------+
| API key missing (cloud)      | Settings drawer auto-opens with focus on key |
|                              | field. Toast: "API key required for Claude." |
+------------------------------+----------------------------------------------+
| User sends while streaming   | Input disabled. Send button replaced with    |
|                              | [STOP] button.                               |
+------------------------------+----------------------------------------------+
| Rate limit hit (cloud)       | Error toast: "Rate limited. Retry in 30s."   |
|                              | Countdown shown.                             |
+------------------------------+----------------------------------------------+
| Very long tool result        | Truncated to 4KB in card. "Show full" button |
|                              | to expand. Always sent fully to LLM.         |
+------------------------------+----------------------------------------------+
```

---

7. Files Summary

TEXT

```
NEW FILES (14)
══════════════

Backend (7):
  api/src/ai/mod.rs
  api/src/ai/types.rs
  api/src/ai/session.rs
  api/src/ai/tool_executor.rs
  api/src/ai/routes.rs
  api/src/ai/providers/mod.rs
  api/src/ai/providers/local.rs

Frontend (7):
  src/t3-react/features/ai-chat/index.ts
  src/t3-react/features/ai-chat/pages/AiChatPage.tsx
  src/t3-react/features/ai-chat/hooks/useAiChatStream.ts
  src/t3-react/features/ai-chat/hooks/useAiSettings.ts
  src/t3-react/features/ai-chat/components/ChatPanel.tsx
  src/t3-react/features/ai-chat/components/ChatMessage.tsx
  src/t3-react/features/ai-chat/components/ChatInput.tsx

MODIFIED FILES (4)
══════════════════

  api/src/lib.rs                          -- add `pub mod ai;`
  api/src/server.rs                       -- merge ai routes
  src/t3-react/app/App.tsx                -- add lazy import + route
  src/t3-react/app/config/toolbarConfig.ts  -- add toolbar button

PHASE 1 SCOPE: 14 new + 4 modified = 18 files
```

---

8. Final Suggestions

8.1 Start with Phase 1 ONLY — get streaming working end-to-end first

The hardest part is the SSE plumbing (backend SSE handler + frontend ReadableStream parser). Get a simple "echo" working: user sends text, backend proxies to Ollama, streams text back. No tools, no settings panel, no sessions. Just:

1. Hardcoded endpoint `http://localhost:11434/v1`

2. Hardcoded model `llama3.1:8b`
3. Simple chat: POST -> stream -> display

Once that works, everything else builds on top.

8.2 Refactor mcp.rs BEFORE building tool_executor

The MCP dispatch in `mcp.rs` is a giant `match method_name { ... }` block. Before Phase 2, extract each arm into a named `pub(crate)` function. This is a pure refactor — no behavior change. It makes `tool_executor.rs` trivial.

8.3 Use a fixed system prompt initially

Don't make the system prompt configurable in Phase 1. Hardcode the building automation assistant prompt. Add the settings UI for it in Phase 4.

8.4 Test with a simple curl first

Before building any frontend, verify the backend works:

```shell
curl -N -X POST http://localhost:9103/api/ai/chat \
  -H "Content-Type: application/json" \
  -d '{"provider":"local","model":"llama3.1:8b","messages":[{"role":"user","content":"He
```

You should see SSE chunks streaming back. If this doesn't work, the frontend definitely won't.

8.5 Tool names exposed to LLM should match MCP exactly

The `name` field in tool definitions sent to the LLM must match the dispatch key in `tool_executor.rs`. Keep a single source of truth — ideally the `TOOLS` lazy_static in `mcp.rs`. Reference it when building the provider request.

8.6 Keep the right sidebar truly optional

The ContextPanel (suggestions, device filter, tool log) is useful but not essential for Phase 1. Defer it to Phase 2+. Phase 1 should be a clean two-element layout: messages + input.

8.7 Suggested test flow for Phase 1

TEXT

```
Install:  ollama pull llama3.1:8b
Run:      ollama serve
Verify:   curl http://localhost:11434/api/tags
Build:    cd api && cargo build
Test:     curl -N ... (see 8.4 above)
Build FE: yarn build   (or dev server)
Navigate: http://localhost:9103/#/t3000/ai-chat
Type:     "Hello!"
Expect:   Streaming AI response in chat bubbles
```

> **Next:** Once this design is approved, start with Phase 1 — Core Plumbing.

FDD — Native Fault Detection & Diagnostics for T3000

> Native Rust FDD engine with DB-managed rules (no SQL engine, nothing embedded).

---

1. Overview & goal

Add a **Fault Detection & Diagnostics (FDD)** capability to T3000 so the platform can proactively detect common HVAC equipment faults from historical trend data, and let the AI chat explain them.

The design deliberately follows open-fdd's **concepts** (rules as tunable data, semantic roles, confirm windows) but re-implemented **natively in T3000's Rust stack**:

- ✅ rules stored as **DB rows** (managed like the existing auto-tagging rules)
- ✅ one reusable **Rust evaluator** (no SQL engine, no DataFusion)
- ✅ nothing compiled into the binary (rules are data)
- ✅ reuses what T3000 already has: Haystack/Brick tags, trendlog history, MCP server, AI chat

---

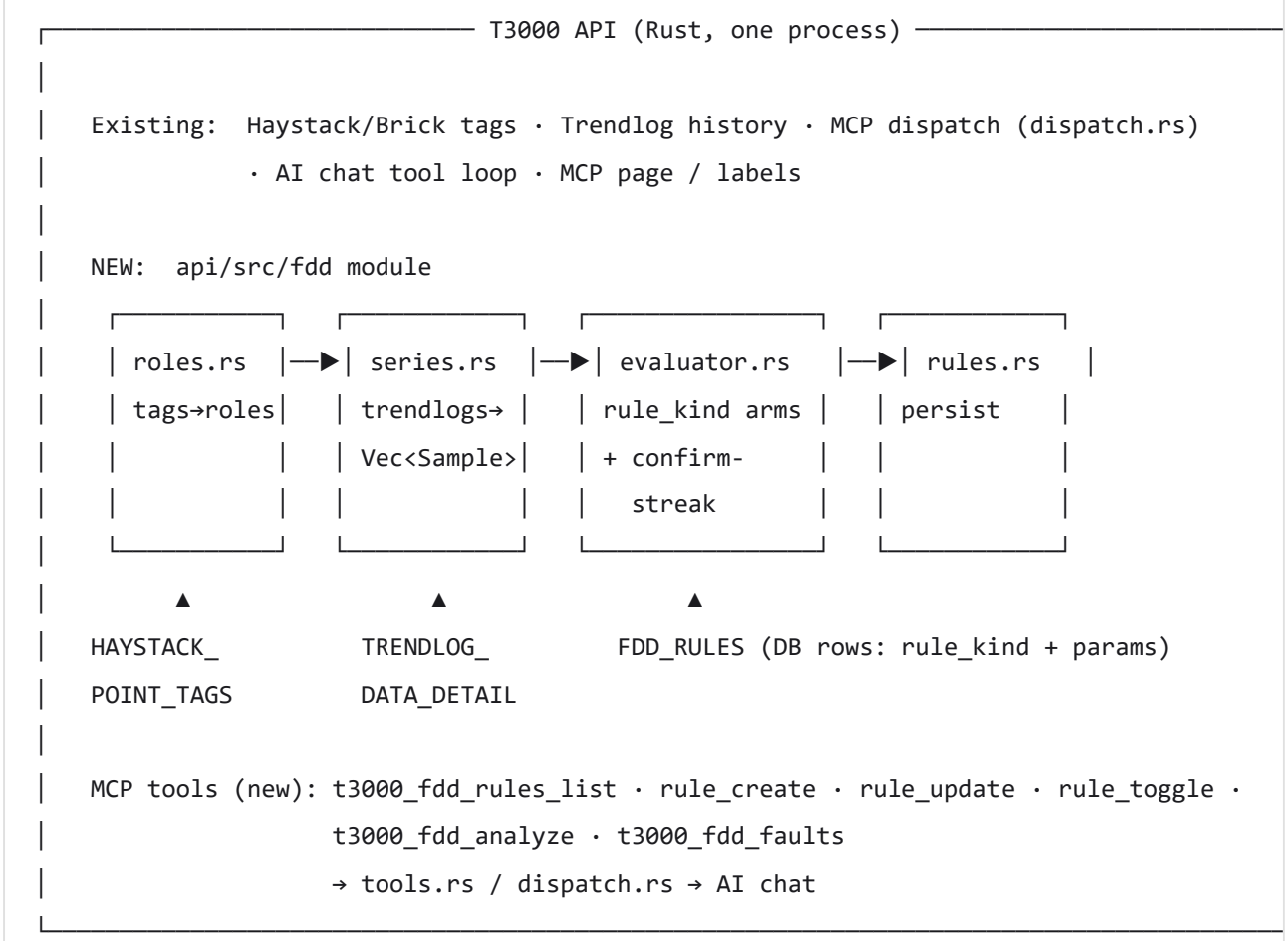
2. Design decisions

Decision	Choice	Why
Engine	**Native Rust evaluator**	no SQL-in-DB, no DataFusion, typed, testable
Rule storage	**DB table `FDD_RULES`**	managed at runtime like `HAYSTACK_AUTO_TAGGING_RULES`; nothing embedded
Role source	**Haystack/Brick tags**	already in T3000 (`HAYSTACK_POINT_TAGS`, `HAYSTACK_POINT_BRICK_CLASS`)
Time-series	**TRENDLOG_DATA_DETAIL**	existing history; pivot to wide samples on demand
Discovery	**not built**	explicit `analyze` calls; scheduled scans are a later phase
Findings	**`FDD_FINDINGS` table**	`t3000_fdd_analyze` persists active faults; `t3000_fdd_faults` lists history

---

3. Architecture

TEXT



4. Data model — `FDD_RULES` table

```SQL
CREATE TABLE IF NOT EXISTS FDD_RULES (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  rule_id        TEXT NOT NULL UNIQUE,    -- 'ECON-4'
  rule_name      TEXT NOT NULL,
  category       TEXT,                    -- 'economizer' | 'sensor' | 'vav' | 'chw' | 'z
  description    TEXT,
  rule_kind      TEXT NOT NULL,           -- which Rust evaluator runs
  required_roles TEXT,                    -- JSON array of semantic roles
  params_json    TEXT,                    -- JSON of tunable params
  severity       TEXT DEFAULT 'warning', -- info | warning | critical
  enabled        BOOLEAN DEFAULT 1,
  created_at     TEXT,
  updated_at     TEXT
);
```

> `FDD_FINDINGS` stores persisted fault detections — written by `t3000_fdd_analyze` and read by `t3000_fdd_faults`. Time-series and tags reuse existing tables.

Seed

The catalog is seeded by the SeaORM migration `m20260819_add_fdd_tables` (fresh DBs) and by `fdd::rules::ensure_schema` on startup (idempotent `INSERT OR IGNORE`, so existing DBs pick up new rules). After seeding the DB is authoritative — rules can be edited/toggled/deleted at runtime with no rebuild.

```SQL
INSERT INTO FDD_RULES (rule_id, rule_name, category, description, rule_kind, required_ro
VALUES
('ECON-4','Low outdoor-air fraction','economizer',
 'Economizer not bringing in enough OA when free-cooling is available',
 'EconomizerOaFraction',
 '["mat","rat","oa_t","fan_cmd"]',
 '{"oa_min_pct":15,"confirm_rows":4,"poll_seconds":300}',
 'warning', 1);
```

---

5. The Rust engine

5.1 Sample & roles

RUST

```rust
/// One "wide" history row built from TRENDLOG_DATA_DETAIL for a device/equipment.
/// Roles are resolved from Haystack tags; values are keyed by role.
pub struct Sample {
    pub ts: String,                      // LoggingTime_Fmt, e.g. "2026-08-19 12:30:00"
    pub values: HashMap<String, f64>,    // role → value, e.g. "oa_t" → 55.0
}
```

`roles.rs` maps each point's **Haystack tags / Brick class** to a role on `Sample`:

TEXT

```
OA_T        → oa_t       MAT         → mat         SupplyTemp → sat
RAT         → rat        ZONE_T      → zone_t      FAN_CMD    → fan_cmd
FAN_STATUS  → fan_status OA_DAMPER   → damper_pct  CHW_S      → chw_s
CHW_R       → chw_r      CHW_DP      → chw_dp      SAT_SP     → sat_sp
```

5.2 The reusable evaluator (the heart)

Every rule = **a per-sample condition** + **a confirm streak** + **a sample interval** → fault hours.

RUST

```rust
/// Generic confirm-window evaluator: count fault only after N consecutive samples.
fn fault_hours<F>(series: &[Sample], mut raw_fault: F, confirm_rows: u32, poll_seconds:
where F: FnMut(&Sample) -> bool
{
    let mut streak = 0u32;
    let mut hours = 0.0;
    for s in series {
        streak = if raw_fault(s) { streak + 1 } else { 0 };
        if streak >= confirm_rows {
            hours += poll_seconds as f64 / 3600.0;
            streak = 0;
        }
    }
    hours
}

/// Read a role's value from a sample (None if the device has no such role).
fn field(s: &Sample, role: &str) -> Option<f64> { s.values.get(role).copied() }

/// Dispatch a FDD_RULES row to the right condition. Returns fault hours + evidence.
pub fn eval_rule(rule: &Rule, series: &[Sample]) -> Finding {
    let confirm = rule.params["confirm_rows"].as_u64().unwrap_or(4) as u32;
    let poll = rule.params["poll_seconds"].as_u64().unwrap_or(300);
    match rule.rule_kind.as_str() {
        "ThresholdAbove" => {
            let f = rule.params["field"].as_str().unwrap_or("");
            let limit = rule.params["limit"].as_f64().unwrap_or(0.0);
            let hours = fault_hours(series, |s| field(s, f).is_some_and(|v| v > limit),
            Finding { fault_hours: hours, evidence: json!({"field": f, "limit": limit})
        }
        "ThresholdBelow" => { /* same, reversed */ }
        "RangeBand"      => { /* value outside [lo, hi] */ }
        "StuckValue"     => { /* no change over a sliding window of `window_rows` (froze
        "FanMismatch"    => { /* (fan_cmd on) != (fan_status on) after 0..100 normalisat
        "EconomizerOaFraction" => { /* OA fraction (mat-rat)/(oa_t-rat) < oa_min_pct whi
        "EconomizerStuckClosed" => { /* damper_pct < 5 while fan on and oa_t < mat */ }
        "ChwLowDeltaT"   => { /* chw_r - chw_s < min_dt */ }
        "SupplyTempDeviation" => { /* |sat - sat_sp| > max_dev */ }
        _ => Finding { fault_hours: 0.0, evidence: json!({}) },  // new rule_kinds = new
```

```
    }
}
```

Adding a rule = one new `rule_kind` arm **or** reuse a generic one + insert a `FDD_RULES` row.

---

6. The complete rule catalog

Starter catalog (16 rules). All are DB rows seeded on first run; every rule is tunable via `params_json` and toggleable via `enabled`.

6.1 Economizer

rule_id	Name	rule_kind	Required roles	Default params	Severity
ECON-1	OA damper stuck closed	`EconomizerStuckClosed`	`oa_t, mat, fan_cmd, damper_pct`	`confirm_rows:4, poll_seconds:300`	warning
ECON-3	Mechanical cooling without economizing	`EconomizerOaFraction`	`mat, rat, oa_t, fan_cmd`	`oa_min_pct:0, confirm_rows:4`	warning
ECON-4	Low OA fraction	`EconomizerOaFraction`	`mat, rat, oa_t, fan_cmd`	`oa_min_pct:15, confirm_rows:4`	warning
ECON-6	Economizer freezing risk	`ThresholdBelow`	`mat`	`field:"mat", limit:2.0, confirm_rows:3`	critical
ECON-7	Not economizing when it should	`EconomizerOaFraction`	`mat, rat, oa_t, fan_cmd`	`oa_min_pct:0, confirm_rows:4`	info

6.2 Fan

rule_id	Name	rule_kind	Required roles	Default params	Severity
CMD-1	Fan cmd/status mismatch	`FanMismatch`	`fan_cmd, fan_status`	`confirm_rows:4, poll_seconds:300`	warning
FAN-RUNTIME	Fan runtime hours (metric)	`ThresholdAbove`	`fan_cmd`	`field:"fan_cmd", limit:0.05`	info

6.3 Supply air sensor

rule_id	Name	rule_kind	Required roles	Default params	Severity
SAT-HIGH	Supply air temp too high	`ThresholdAbove`	`sat`	`field:"sat", limit:100, confirm_rows:4`	critical
SAT-LOW	Supply air temp too low	`ThresholdBelow`	`sat`	`field:"sat", limit:40, confirm_rows:4`	critical
SAT-DEV	Supply air temp deviates from setpoint	`SupplyTempDeviation`	`sat, sat_sp`	`max_dev:5, confirm_rows:4`	warning
SAT-STUCK	Supply air temp sensor frozen	`StuckValue`	`sat`	`deadband:0.1, window_rows:12`	warning

6.4 Zone / VAV

rule_id	Name	rule_kind	Required roles	Default params	Severity
VAV-1	Zone comfort band violation	`RangeBand`	`zone_t`	`lo:70, hi:75, confirm_rows:4`	warning
ZONE-STUCK	Zone temp sensor frozen	`StuckValue`	`zone_t`	`deadband:0.1, window_rows:12`	warning

6.5 Chilled water

rule_id	Name	rule_kind	Required roles	Default params	Severity
CHW-1	Low delta-T across coil	`ChwLowDeltaT`	`chw_s, chw_r`	`min_dt:5, confirm_rows:4`	warning
CHW-2	CHW supply pressure low	`ThresholdBelow`	`chw_dp`	`field:"chw_dp", limit:8, confirm_rows:4`	warning
CHW-3	CHW supply temp out of band	`RangeBand`	`chw_s`	`lo:40, hi:48, confirm_rows:4`	warning

> The catalog grows by **inserting rows** (and, rarely, adding a `rule_kind` evaluator arm).

---

7. Management (same as auto-tagging)

Action	MCP tool	DB effect	Like
List rules	`t3000_fdd_rules_list`	`SELECT * FROM FDD_RULES`	`t3000_haystack_list_rules`
Create custom rule	`t3000_fdd_rule_create` (confirm)	`INSERT`	`t3000_rule_create`
Edit rule / tune params	`t3000_fdd_rule_update` (confirm)	`UPDATE params_json`	—
Enable / disable	`t3000_fdd_rule_toggle` (confirm)	`UPDATE enabled`	`t3000_rule_toggle`
Backup / share	`t3000_fdd_rule_export` → JSON	read	—
Restore	`t3000_fdd_rule_import` (confirm)	upsert	—

Tool input example — `t3000_fdd_rule_toggle`:

```JSON
{ "rule_id": "ECON-4", "enabled": false, "confirm": true }
```

Tool input example — `t3000_fdd_analyze`:

```JSON
{
  "serial_number": 123,
  "equipment": "AHU-1",
  "range_hours": 24,
  "rules": ["ECON-4", "SAT-HIGH"]
}
```

---

8. Findings output

```JSON
{
  "device": 123,
  "equipment": "AHU-1",
  "range_hours": 24,
  "findings": [
    {
      "rule_id": "ECON-4",
      "rule_name": "Low outdoor-air fraction",
      "severity": "warning",
      "fault_hours": 1.5,
      "evidence": { "oa_min_pct": 15, "samples": 288, "min_oa_frac": 3.2 },
      "suggestion": "Check OA damper actuator and economizer control."
    }
  ]
}
```

---

9. AI chat integration

`t3000_fdd_*` tools join `TOOLS` → `get_all_tool_defs()` automatically. A user asks:

> "Why is AHU-1 faulting?"

The AI calls `t3000_fdd_analyze {serial_number:123, equipment:"AHU-1", range_hours:24}` → gets findings → explains in plain language + suggests fixes (optionally via `t3000_doc_read` / `t3000_device_diagnostics`).

Frontend additions (same pipeline as existing tools):

- `useAiChatStream.ts` tool labels (e.g. `t3000_fdd_analyze: 'Running fault detection…'`)
- `McpServerPage.tsx` tool-table rows + Examples prompts

---

10. Testing

- **Unit** (no DB): build synthetic `Vec<Sample>`, run `eval_rule` per `rule_kind`, assert `fault_hours` (streak/confirm behaviour).
- **Integration** (MCP test suite, like `device_operations.rs`): tool existence + schema; DB test seeds `FDD_RULES`, runs `t3000_fdd_analyze` against a small synthetic series inserted into `TRENDLOG_DATA_DETAIL`.
- **Tool-count** test: bump the count in `tool_definitions.rs` when tools are added.

---

11. Known limitations

- **Tag coverage** — rules only fire where auto-tagging produced the required roles. `t3000_fdd_analyze` returns `roles_found` per analysis so gaps are visible.
- **Trendlog resolution** — low-resolution logs limit detection quality; `poll_seconds` in each rule must match the actual trendlog log interval for accurate fault hours.
- **Rule correctness** — the starter catalog is heuristic; thresholds should be validated and tuned against real site data.

The OpenBIM and Haystack plan

1. Executive Summary & Objective

The goal of this initiative is to transition the Temco Controls software ecosystem (**T3000**) from a legacy configuration utility into a modern, open-standard **Building Operating System (BOS)**. By integrating **OpenBIM** standards (IFC via **Bonsai BIM** and **thatOpen**) and **Semantic Tagging** (Project Haystack) directly into our core application architecture, we will provide an automated, zero-configuration solution designed specifically for high-density, mission-critical infrastructure clients like **PLC Group** (Data Centers/Telecom).

High-Level Architectural Flow

1. **Physical/Spatial Input:** Bonsai BIM (Desktop Authoring via Blender Engine) builds/modifies a completely open, native ISO graph database (`.ifc`).
2. **Web Rendering & Asset Tracking:** `thatOpen` framework streams and renders high-density geometry and structural data directly inside browser-based environments via WebAssembly.
3. **Semantic Layer Integration:** Project Haystack tags are stored *onboard* the hardware non-volatile storage (NVS) and synchronized through the local database.
4. **Autonomous Execution:** Central AI engines (e.g., PLC's *AxIn*) ingest the resulting structured data model via a standardized, firewall-friendly REST/JSON API.

---

2. Component Mapping & Repositories

This plan targets the convergence of two primary Temco Controls open-source development assets:

- **`temcocontrols/T3000Webview`** : The modern web-centric front-end built on Quasar/Vue, supported by a high-performance local Rust service API.
- **`temcocontrols/T3000_Building_Automation_System`** : The mature, core desktop application engine running on C++/MFC.

---

3. Phased Implementation Roadmap

Phase 1: Database & Firmware Extension (The Foundation)

To achieve true onboard edge autonomy, metadata must live as close to the physical I/O as possible.

- **Local Storage Schema Update:** Modify the embedded SQLite schema managed by the `T3000Webview` Rust API to add a dedicated `haystack_tags` string field for all points, variables, and registers.

- **Firmware Modbus/BACnet Mapping:** Allocate a section of the **64KB NVS partition** on the **ESP32-S3** hardware controllers (adjacent to the Matter credential slots) to hold compressed tag strings.
- **C++ Core Integration:** Update the `T3000_Building_Automation_System` C++ engine to serialize and push these tag strings directly to the physical devices during standard programming updates.

Phase 2: WebUI Evolution & Smart Automation (T3000Webview)

Leverage web spreadsheet components to eliminate the engineering bottleneck of manual data configuration.

- **The Semantic Grid:** Integrate a "Haystack Tags" column into the Quasar grid layout (`localhost:3003/#/modbus-register`).
- **Heuristic Matching Engine ("Auto-Tagging"):** Build a client-side JavaScript utility within the web app that parses user descriptive names. For example, a point named `DAT_01` automatically resolves and applies the following semantic tags: `point` , `air` , `discharge` , `temp` , `sensor` .
- **Spatial Canvas Drawer Integration:** Enhance the visual configuration environment. When a technician drags and drops a temperature sensor inside the visual bounding box of an Air Handling Unit (AHU), the interface must automatically append the structural relational tag (e.g., `equipRef: "AHU-1"`) directly into that point's metadata array.

Phase 3: The Inter-Process Communication Loop

Maintain strict state alignment between the C++ native window wrapper and the embedded web container.

- **Web-to-Native Event Push:** When changes are made inside the `T3000Webview` grid, dispatch a structured JSON message across the existing messaging bridge:

```JSON
{
  "action": "update_tags",
  "panelId": 144,
  "entryIndex": 1,
  "tableType": "INPUT",
  "tags": "sensor temp air discharge unit:"°C""
}
```

- **Native Interception:** Update the C++ window message handler in `T3000_Building_Automation_System` to capture this event, commit the update to the local system `.db` / `.prog` file, and stage the controller configuration block for transmission.

Phase 4: OpenBIM Engine Integration (Bonsai BIM + thatOpen)

Transition from static 2D isometric graphics to a native, interactive 3D informational model.

- **Authoring & Asset Modeling (Bonsai BIM):** Utilize the desktop-level Python/Blender environment to map critical infrastructure assets (Server Racks, CRAC Units, DC Power Plants, UPS Battery Strings) using native IFC standards.
- **Web Stream Pipeline (thatOpen):** Integrate the web-native `@thatopen/components` framework into `T3000Webview`. This allows the application to stream, parse, and render compressed web-IFC geometry fragments directly inside a browser tab at native desktop speeds.
- **Semantic Binding:** Link the `GlobalId` (GUID) of the physical IFC object explicitly to the corresponding Haystack `equipRef` or `point` record in the database.

Phase 5: Enterprise REST API Layer (The Open Handshake)

Enable a direct data pipeline to cloud analytics platforms, completely bypassing heavy UI management steps.

- **Expose External Network Interfaces:** Configure the lightweight Rust microservice embedded within `T3000Webview` to safely bind to external network adapters, expanding past localhost constraints.
- **Standardized Haystack Operations:** Build endpoints compliant with the core Project Haystack REST protocol:
  - `POST /api/haystack/read` -> Queries the SQLite points table using standard semantic tags.
  - `POST /api/haystack/hisRead` -> Extracts raw data history and trend logs directly from the local file system.
- **Interactive Documentation (Swagger / OpenAPI):** Mount an interactive `/swagger` interface on the Rust microservice. This allows enterprise engineering teams to immediately discover endpoints, run validation scripts, and integrate data straight into modern AI engines.

---

4. Key Target Verticals & Specialized Templates

Data Centers (High-Density Power & Thermal Asset Sets)

- **Server Racks:** `rack`, `equip`, `it`, `power`, `pdu`, `temp`
- **CRAC Precision Cooling Units:** `hvac`, `cooling`, `ahu`, `inlet`, `exhaust`, `fan`, `temp`
- **UPS Systems:** `elec`, `pdu`, `ups`, `voltage`, `current`

Telecommunications (Dispersed Remote Edge Infrastructure)

- **DC Power Plants:** `elec`, `dc`, `plant`, `voltage`, `current`, `charge`
- **Battery Banks:** `battery`, `dc`, `equip`, `temp`, `voltage`, `health`

---

5. Strategic Competitive Advantage

By combining open-source design tools with firmware-level semantic tagging, Temco Controls bypasses the closed licensing barriers and specialized technician dependencies typical of legacy vendors (such as Honeywell Forge or Johnson Controls OpenBlue). The data from our hardware remains entirely open, self-describing, and ready for automated integration directly into the customer's choice of advanced analytics platforms from day one.